LEAD GEN

Lead Gen Agent

The Complete Plain-English Guide

What it is, how it was made, how to set it up and run it, everything it can do, and every tool it connects to.

Iksula Agentic Org · iKshana assembly line
Plain-English guide · no technical background needed
Version 2026-06-23

Contents

1 What it is, and where it fits in the bigger picture

What this agent is, in one sentence

The **Lead Gen agent** is the part of Iksula's AI workforce whose job is to turn *interest* into a *booked, accepted sales meeting*. Someone comments on a LinkedIn post, fills in a form, downloads a guide, or shows up on an event attendee list — that is "interest". Lead Gen takes that raw interest and walks it all the way to "a salesperson has a meeting on their calendar with a real, qualified buyer."

In Iksula's own words from its instruction file (the "Mandate"), the agent's purpose is to:

capture interest from every channel, qualify it once and consistently, nurture the not-yet-ready, orchestrate outreach against target accounts, and convert it to a booked, accepted meeting — handed to the right owner.

A useful analogy: think of a restaurant. The marketing team out front gets people *interested* and walking past the door. Lead Gen is the **host at the door** — it greets each person, figures out if they are a genuine customer (not a competitor, not someone already eating inside), seats the right ones, and walks them to the right table. It does **not** cook the food and it does **not** take the order — it gets the right guest to the right table, then hands over.

What it actually does (the day-to-day)

Stepping through it in plain order, Lead Gen:

- **Sources prospect lists** — pulls lists of likely buyers from tools like **Apollo** (a contact database — filtered lists with emails and phone numbers), **BuiltWith** (a tool that tells you what technology a company's website runs, e.g. "all Adobe partners"), and **LinkedIn / Sales Navigator** (manual people-and-company discovery for regions where Apollo is thin). It can also pull **event attendee and speaker lists**.
- **Cleans and de-duplicates the lists** — fixes mismatched rows (a name lined up with the wrong company or email), drops bad data, and cross-checks Iksula's customer database so it does not accidentally target an **existing client** or a person another salesperson is already talking to. Important rule: the agent **highlights** these rows and removes them **only after a human says yes** — it never silently deletes, because it might be wrong about who is a client.
- **Scores each lead** — grades the *fit* (is this the right kind of company and person?) and the *intent* (are they actually showing buying behaviour?) on two separate scales, then decides jointly. A poor-fit company never becomes a "qualified" lead no matter how many emails it opens.
- **Builds email campaigns in Woodpecker** — Woodpecker is the email outreach engine (it builds sequences of follow-up emails and handles replies). The agent **creates** new campaigns but is deliberately built so it **cannot press "send"** (more on this below).
- **Handles replies and reporting** — routes the people who reply or click, and reports the funnel numbers (how many leads, how many meetings) upward.
- **Routes qualified leads to Sales** — packages a "context pack" (who the buyer is, why they fit, why now) and hands a meeting-ready lead to the correct salesperson.

What it deliberately does **NOT** do (this matters — it keeps the agents from stepping on each other):

- It does **not** write the content or marketing copy (that is the Content Creator).
- It does **not** post publicly on LinkedIn or run community engagement (that is the Growth Hacker — on Lead Gen's side, LinkedIn is strictly 1-to-1 messaging, never posting).
- It does **not** plan the advertising budget (that is the Media Planner).
- It does **not** close the deal — it stops at the *booked, accepted meeting* and hands over to a human salesperson.
- It does **not** decide on its own that a lead is "qualified" — the receiving salesperson has to accept it as real.

Who it is for

This agent is for **Iksula's own sales growth** and, in everyday terms, it is run by an **operator** — today that is mainly **Vishal**, who runs the real-world version of this exact workflow by hand (sourcing in Apollo, building campaigns in Woodpecker, cleaning data, diffing open-rate reports). The agent is being built to take the heavy, repetitive parts off his plate while keeping the judgement calls — copy approval, "yes you may delete this row", "yes you may send" — firmly with a human. **DJ** (the founder) sets the strategy and signs the legal go-aheads; **Yatin** builds the agent.

The iKshana assembly line — and where Lead Gen sits

Iksula is building an AI-native marketing-and-sales organisation called **iKshana**: a chain of specialist AI agents, each doing one job and handing off to the next, like a factory assembly line. The full chain:

#	Agent	Its job, in plain words
1	RBM	Strategy / what business we are going after
2	Solution Architect	Shapes the offer/solution
3	Thought Leadership	Decides the point of view and themes
4	Media Planner	Plans where content runs / channel spend
5	Content Creator	Actually writes/makes the content
6	**Growth Hacker**	Publishes content, runs the channels, and **surfaces interest** (who reacted, clicked, commented)
7	**LEAD GEN ← this agent**	**Captures that interest, qualifies it, nurtures it, and converts it into a booked meeting**
8	Sales / KAM	Takes the meeting and closes the deal

Lead Gen's own file describes its place as **"the back half of the Commercial fast loop"**: the content engine (steps 1–6) *creates and surfaces demand*; **Lead Gen captures, qualifies, nurtures, and converts it.** It is *fed by* the Growth Hacker (online interest) plus three other channels — **Events, Inside Sales, and Reference** (referrals). It *feeds* two places: **Sales / New Client Acquisition and Key Account Management** (the qualified leads), and the **Brain** (just the summary numbers).

One nice detail in how it routes: **new** companies go to Sales / New Client Acquisition, but an **existing client** that shows fresh interest goes to **Key Account Management** as an expansion opportunity — never as a cold pitch. You do not cold-email a company that is already your customer.

Brain / Spine / Hands / Zoho — the everyday-office analogy

iKshana is organised into four parts. The simplest way to picture them is a **shared office**:

- **The BRAIN** — a Google Drive folder called `_brain/`. This is the company's permanent **reference library**. It holds knowledge and *aggregate numbers only* (e.g. "we got 200 content-sourced leads in May, 30% became meetings"). It is **append-only** (you add pages, you never erase) and — crucially — it contains **no personal data about any individual**. Think of it as the library where you file final, published reports. Lead Gen *reads* things like `icp-audience` (who counts as a good-fit buyer) and *writes* one summary feed named `performance-analytics-leadgen-YYMMDD` — numbers only, never a list of people.
- **The SPINE** — another Drive area, `_spine/`. This is the **shared in-tray and to-do board** between agents. Unlike the library, it **can change**. It holds the work-in-progress hand-off records and queues. This is where the Growth Hacker drops "I found interest" notes and where Lead Gen picks them up.

- **The HANDS** — the agents themselves, the ones that actually *do* things. Lead Gen is a Hand.
- **ZOHO CRM** — the **system-of-record for individual people**. (CRM = Customer Relationship Management software; it is the master file cabinet for contacts.) This is the live, **changeable** record for each prospect: their contact details, their score, whether they have a lawful basis to be emailed, whether they have opted out, what stage they are at. The Brain can't hold this (it is frozen and personal-data-free), so the per-person, ever-changing facts live in Zoho instead. Iksula's Zoho lives in the US data centre, org `29004087`.

So the division of labour is: **the Brain is the library** (permanent, anonymous summaries), **the Spine is the shared to-do board** (changeable hand-offs), **the Hands are the workers**, and **Zoho is the customer file cabinet** (the live, personal, per-person truth). Lead Gen is the only agent that touches all four.

How each agent is actually built — Skill + Toolkit

Each iKshana agent is made of two pieces, and it is worth understanding this because it is what makes the safety real, not just wishful:

- **A SKILL** — a markdown instruction file (`SKILL.md`) that the AI reads and follows. It is the written job description and rulebook.
- **A TOOLKIT** — actual Python code (under `tools/leadgen/`) that **enforces** the dangerous rules so the agent literally *cannot* do the forbidden thing. For example, the toolkit's email client (`wp_client.WoodpeckerClient`) is **create-only** — there is no "send"/"run" command in it at all, and its internet access is allow-listed to only "list campaigns" and "create campaign". The check that decides whether anyone may be emailed (`presend_gate.evaluate()`) is a hard wall that returns ALLOW or HALT — and today it always returns HALT with zero eligible people. So the rule isn't just *written*, it's *built in*.

"The seam" — the one hand-off you need to understand

The most important hand-off in this whole picture is the single connection point between the **Growth Hacker** and **Lead Gen**. Iksula calls it **"the seam"**.

In plain terms: the Growth Hacker is the agent out on the public channels noticing that *somebody showed interest* — a comment, a reply, a form fill, a click. When it notices one, it drops a small note onto the shared Spine to-do board. Each note is a record called a `content-sourced-lead`. That stream of notes **is** the seam.

The clean rule of the seam is: **the Growth Hacker emits raw, unqualified interest events; Lead Gen owns absolutely everything that happens after.** The note the Growth Hacker leaves carries **no score and no decision** — just the raw facts it observed:

- a `correlation_id` (a unique ticket number for this one interest event, so it can never be processed twice),
- whatever it saw about the person (`contact_identity` — maybe just a LinkedIn handle, maybe a volunteered email, sometimes only an anonymous click),
- which post/asset sparked it (`source_asset_ref`),
- which surface it happened on (`source_channel`, e.g. `LinkedIn-post`, `Blog-form`),
- the raw action itself (`intent_signal`, e.g. "commented", "downloaded"),
- the person's best-known `region` and a `lawful_basis_tag` derived from it (a flag the Growth Hacker *stamps* but **Lead Gen enforces**).

Then Lead Gen does all the real work: de-anonymise and enrich the person, de-duplicate them into one clean Zoho record, check there is a lawful basis to contact them, scrub them against the suppression list (opt-outs, competitors, existing clients), score them, nurture them, and eventually convert them. When it has consumed a note, it writes a small **ACK** ("received / de-duped / rejected") back onto that same Spine record so the same interest is never worked twice.

Two everyday points about the seam:

- **It is a Spine record, not a Brain entry.** Because the note is about a *specific person*, changes the moment it is touched, and is used for look-ups — that is exactly the kind of live, personal data the Brain (the frozen, anonymous library) cannot hold. Only the anonymous count ("how many content-sourced leads came in") ever flows up to the Brain.
- **Lead Gen consumes the seam; it never goes back and re-scrapes the public posts.** Once the Growth Hacker has reported an interest event, Lead Gen trusts the note and works *forward* — it never re-detects what the Growth Hacker already

surfaced.

The honest reality today (2026-06)

Everything above is **built and safety-checked, but not yet running live.** Three things are worth stating plainly:

- **No emails go out today — by design.** Woodpecker is a **live production account** with 97 real campaigns (3 actively sending). The Lead Gen agent is treated as a *guest*: it can **create** new campaigns (tagged `[LG-AGENT]`) and record each one in a ledger, but it **never** presses send and **never** touches a campaign it didn't create.
- **"Lawful basis" is set nowhere yet.** Lawful basis means *recorded permission/legal grounds to email a specific person*. In Iksula's Zoho, that field (`Data_Processing_Basis`) is currently empty for 100% of leads. The rule is: no lawful basis → that person is not emailed and not even loaded into Woodpecker. So **the correct number of people to email today is zero**, until DJ signs the legal go-ahead (the LIA, or Legitimate Interest Assessment) and a properly warmed-up sending mailbox is provided.
- **Copy stays human-gated.** The agent can draft outreach, but a human refines and approves it, and a human triggers the very first send. The agent proposes; people decide.

In short: Lead Gen is the host-at-the-door of Iksula's AI sales factory — it takes interest the Growth Hacker surfaces through the seam, qualifies it carefully, stages (but never sends) the outreach, and walks meeting-ready buyers to the right salesperson — all while the riskiest actions stay locked behind a human's "yes."

2 How it was made

The Lead Gen agent is built from two parts that work together. Think of it like a careful new employee (the part that thinks) plus a locked toolbox (the part that acts). The thinking part follows written instructions. The toolbox part is wired so that the dangerous tools simply are not in it.

Part 1 — The SKILL: plain-English instructions the AI reads

A "skill" here is just a long, carefully-written instruction document in plain English (technically Markdown, a simple text format). The AI reads it at the start of every run and follows it, the same way a new hire reads a standard-operating-procedure binder before touching anything.

For Lead Gen, the main instruction file is:

- `iksula-marketplace/plugins/iksula-agents/skills/lead-gen/SKILL.md`

It spells out the job step by step: take in prospect interest, clean and de-duplicate it, score it, draft outreach, build email campaigns in "create-only" mode (build the campaign but never press send), and route qualified leads to Sales. It also bakes in the rules that keep Iksula safe — for example: never cold-email an existing client, only email people for whom permission has been recorded, and never auto-send at scale.

The SKILL leans on a few supporting reference documents (also plain English), each covering one area in depth:

Reference file	What it explains
`references/sending-stack.md`	The email tools (Woodpecker for cold, Mailerlite for opted-in) and the hard "never press send" rules
`references/seam-and-compliance.md`	How leads arrive, and the permission/suppression/geography rules
`references/scoring-framework.md`	How a lead is scored on fit and intent
`references/vocabularies.md`	The fixed lists of allowed labels (lead stages, statuses, channels, etc.)

Here is the important limitation of instructions: they are only a promise. An instruction that says "never press send" works only as long as the AI remembers and chooses to obey it. That is not good enough when the email account is a LIVE one with 97 real campaigns. So the instructions are backed by a second part that does not rely on memory or good intentions.

Part 2 — The TOOLKIT: small programs that ENFORCE the rules

The toolkit is a set of small Python programs (Python is a common programming language) kept right next to the instructions, in:

- `iksula-marketplace/plugins/iksula-agents/tools/leadgen/`

The toolkit's whole purpose is to turn the safety rules from *promises the AI must remember* into *physical walls in the code*. The README states it plainly: the goal is to make the safety rails "structural — not a promise the agent must remember, but code that physically cannot send."

The main pieces:

Program (module)	What it does
`config.py`	Holds the hard settings: the on/off kill-switch, the list of allowed mailboxes, and the only web addresses the agent is permitted to call. No secret keys are written in it — they come from the environment only.
`presend_gate.py`	"The wall." Before anything goes out, it checks every rule and returns either ALLOW or HALT. It only decides; it never sends.
`merge_preview.py`	Catches a classic embarrassment: a personalization blank like `{{company}}` that did not get filled in and would print literally to everyone. It flags those rows before any send.
`wp_client.py`	The connector to Woodpecker (the email tool). It can READ the campaign list and CREATE new draft campaigns — and nothing else.
`ledger.py`	An append-only list of every campaign the agent itself created. This list is the ONLY definition of "a campaign the agent owns."
`audit.py`	An append-only "who ran what" record, for accountability.
`preflight.py`	A safe dry-run: an operator can test "would this batch be allowed to send?" and it prints PASS or HALT without creating or sending anything.

"Safe by construction" — there is no send button to press

The cleanest example is the Woodpecker connector, `wp_client.py`. The code comment says it best: this connector "deliberately has NO method that starts, pauses, resumes, stops, or deletes a campaign. The only way to send is a verb that does not exist here."

In plain terms: imagine a TV remote with no power button — not hidden, just never installed. You physically cannot turn the TV on with it, no matter how hard you try. The Lead Gen toolkit is built the same way. The "send" command (`/run`) was never wired in. So even if the AI somehow decided to send, there is no lever in the code to pull.

This is enforced in several overlapping ways, so a slip in one place is caught by another:

- **An allow-list of exactly two actions.** In `config.py`, `ALLOWED_HTTP` lists the only two web requests the agent may ever make: read the campaign list, and create a draft campaign. Any other request — including a hand-built "set this campaign to RUNNING" — is refused before it leaves the building.
- **A blocked-verb list.** `FORBIDDEN_VERBS` in `config.py` names the dangerous actions (`run`, `start`, `resume`, `pause`, `stop`, `delete`). If the agent even tries to call one by name, the code raises a `ForbiddenAction` error and stops. Casing tricks (like `RUN` vs `run`) are handled too.
- **A master kill-switch.** Unless an environment setting called `WOODPECKER_AGENT_BUILD` is explicitly set to `on`, the agent makes zero write calls of any kind. Off by default.
- **Mailbox isolation.** The code refuses to use the company's primary `iksula.com` mailbox or any real person's mailbox (Vishal's, Sam's, etc.). It will only use a separately-provisioned, "warmed" practice mailbox — and right now none has been set up, which by itself blocks building.
- **"Owned" means in the ledger, not the name.** The agent may only touch campaigns it personally created and recorded in `ledger.py`. The other 97 live campaigns are strictly look-but-don't-touch.

Why the correct answer today is ZERO

The wall in `presend_gate.py` only says ALLOW if EVERY one of six conditions is true: the kill-switch is on, the prospect list is a vetted clean sheet (not raw CRM data), the sending mailbox is an approved warmed one, the personalization preview is clean, a human has approved, AND at least one person on the list has recorded permission to be emailed.

Today, no contact in Zoho (the customer database) has that permission recorded yet, and no warmed mailbox exists. So the wall returns HALT and the eligible-to-email count is **zero**. The README is blunt that this is "correct and intended," not a bug. Even the enrollment step (just adding someone to a draft campaign) counts as writing personal data into an outside system, so it re-runs all the same checks per person — and refuses live writes entirely in this build.

It was stress-tested, fixed, and re-checked

This is not just self-graded. The toolkit ships with 24 automated safety tests that anyone can run with one command:

```
python -m unittest leadgen.tests.test_safety
```

These tests confirm the dangerous actions really are impossible — for example, that trying to send raises an error, that a record without permission is rejected, and that a non-owned campaign cannot be touched.

On top of the tests, the toolkit was reviewed by independent "red-team" AI reviewers — reviewers whose job is to attack the design and find a way to make it misbehave. The first review found real weak spots; those were fixed; a second review confirmed the fixes held. The SKILL records the bottom line: the safety tests all pass and the toolkit was "adversarially verified SOUND." In everyday language: outside reviewers tried to break it, the gaps were fixed, and the locks held.

Where to look if you want proof

You do not need to read code to trust this, but if you (or an auditor) want the receipts, these are the load-bearing files:

- The plain-English contract: `iksula-marketplace/plugins/iksula-agents/skills/lead-gen/references/sending-stack.md`
- The toolkit overview: `iksula-marketplace/plugins/iksula-agents/tools/leadgen/README.md`
- The "no send button" connector: `iksula-marketplace/plugins/iksula-agents/tools/leadgen/wp_client.py`
- The decision wall: `iksula-marketplace/plugins/iksula-agents/tools/leadgen/presend_gate.py`

The short version: the instructions tell the agent the right thing to do, and the toolkit makes the wrong thing physically impossible. That belt-and-suspenders design is why this agent can be developed against a live, revenue-generating email account without risking a single accidental email.

3 Everything it can do — with real examples

Think of the Lead Gen agent as a tireless, rule-obsessed sales-development assistant. Its one job is to turn *interest* (someone clicked, replied, downloaded, or attended) into a *booked, accepted sales meeting* — and to do it cleanly enough that Iksula never embarrasses its brand or breaks a privacy law. It does almost everything an inside-sales operations person does, EXCEPT the two things it is forbidden to do: it never writes the marketing content, and it never presses "Send" on a real email. (More on that wall below — it is switched off on purpose today.)

You start it by typing a plain instruction in Claude Code, for example "run lead gen", "qualify these leads", "score this lead", "build the Woodpecker sequence", or "load the leads into Woodpecker". Here is the full menu of what it can do, grouped by the stage of the funnel.

A quick note on three words you'll see throughout:

- **Lawful basis** = a recorded, legal reason to email a particular person (e.g. they consented, or we have a "legitimate interest"). In Zoho (the customer database) this is a field called `Data_Processing_Basis`. Today it is blank on every lead, so the right number of emails to send is **zero**.
- **Suppression** = a do-not-contact list — opt-outs, competitors, and anyone who is already our client.
- **Buying group / committee** = the several people at one company who jointly decide on a purchase (the budget-holder, the internal champion, etc.). The agent scores the *group*, not one loud individual.

1. Sourcing prospect lists (building the raw list)

What it does: Takes a target brief and builds a list of companies and people to approach — pulling from Apollo (a prospect database with emails/phones), BuiltWith (a tool that tells you what technology a website runs on), LinkedIn / Sales Navigator (manual people-discovery), and event attendee/speaker lists.

Example you'd type: *"Run lead gen. Target: Adobe Commerce partners in the Americas. Personas: Head of Ecommerce, Head of Analytics, CDO."*

What it produces: A sourced list. For a technographic target like "Adobe partners" it reaches for BuiltWith (because Apollo can't filter by technology); for a normal firmographic pull it uses Apollo; for a region where Apollo is thin it falls back to LinkedIn. Important detail from Vishal's real workflow: the **filters are applied inside the Apollo tool itself, not inside Claude** — Claude is poor at filtering big datasets, and Apollo charges credits (roughly 45,000 credits per $1,000; email = 1 credit, phone = 8–9, full enrichment = 20–25). So the agent is built to pull emails sparingly and not burn credits dry. Sourcing region is **global** — it is not limited to the US.

2. Consuming "content-sourced leads" from the Growth Hacker (the seam)

What it does: The Growth Hacker agent (which runs Iksula's social posting and community engagement) drops raw "someone showed interest" events into a shared queue called the **seam** (a hand-off table in the shared `_spine/` folder). Lead Gen picks these up. Each event is a single observed action — a LinkedIn comment, a DM, a blog form-fill, a tracked link-click — with no score and no decision attached. Lead Gen owns everything that happens after.

Example you'd type: *"Work the content-sourced leads."*

What it produces: For each event it reads the record verbatim, then writes an **ACK** (acknowledgement) back onto it — marked `received`, `de-duped`, or `rejected` (with a reason). This is **idempotent**, meaning if the same event arrives twice (keyed on a unique `correlation_id`), the second one is simply ignored — no double-counting. (Reference-channel leads — people referred by an existing relationship — do NOT come through this seam; they enter pre-qualified.)

3. De-duplicating into one clean record

What it does: Merges duplicates so the same human isn't worked as three different leads. The unique key is the normalized email address (lowercased, trimmed). If there's no email yet (an anonymous click or a bare LinkedIn handle), it uses a composite key (platform + handle, or a click ID) and then promotes to the email key once enrichment finds an email — merging the duplicates into one record.

Example you'd type: *"De-duplicate this Apollo pull against Zoho and flag anyone we already know."*

What it produces: A single canonical record per person, plus — and this is a hard rule from Vishal — a **highlighted list of suspected duplicates / existing clients / colleague-owned leads that it removes ONLY after you say yes.** It never silently deletes a row, because it could be wrong about who is a client.

4. De-anonymizing website visitors — geo-gated

What it does: Turns an anonymous website visitor into a named person/company — but only where that is legal. This is **geography-gated**: turning a visitor into a *named individual* is **US-IP-only**. For Europe, India, and everywhere else it can only go to the **company level** (under GDPR in Europe and the DPDP Act in India). Note a subtlety baked into the Zoho rules: the CRM region value `AMERICAS` is not the same as "USA", so person-level de-anon also requires a country check.

Example you'd type: *"De-anonymize this week's visitors and enrich to ICP."*

What it produces: Enriched person/company records *for US visitors only*; company-level-only records for everyone else; and a hard stop (the record goes `ON_HOLD`) for anyone where there's no lawful basis. There is even a safety expectation that an automated test proves zero person-level resolution ever fires on an EU or India IP address.

5. Enriching records (firmographic + technographic)

What it does: Fills in the missing business facts — industry, company size, revenue/GMV band, ecommerce platform, marketplace presence, the tech stack — each with a note of *where the fact came from* (per-field provenance).

Example you'd type: *"Enrich the cleared accounts and show me where each data point came from."*

What it produces: Fuller records, with a source tag on each appended field. It uses Zoho's existing `Enrich_Status__s` field (values like `Available`, `Enriched`, `Data not found`) to track enrichment state.

6. Two-axis scoring and the MQL gate (the heart of qualification)

What it does: Scores every prospect on **two separate axes** and never collapses them into one number:

- **Fit (the WHO)** — graded A / B / C / D from firmographics + technographics + role. Computed at the account level.
- **Intent/Engagement (the WHAT-THEY-DO)** — a points score from real behaviour (pricing-page views, demo requests, replies, clicks), time-decayed so old "zombie" interest fades.

The two gate **jointly**. Three iron rules: a **grade-D** prospect never becomes an MQL (Marketing-Qualified Lead) no matter how hot; **email opens count for ~zero** (Apple Mail Privacy fakes ~50% of opens); and engagement is **rolled up to the whole buying committee**, so a lone champion's clicks can't manufacture an MQL while the budget-holder is silent (a "single-threaded" account is capped below MQL).

Example you'd type: *"Score this lead and show me the why."* or *"Qualify these 200 leads."*

What it produces: For each account, a fit grade (A/B/C/D), an engagement score, a joint MQL verdict, and — crucially — a readable **"why" breakdown** listing the contributing signals and points. Every score is explainable; the agent never produces a black-box number.

The exact weights, point values, grade-band cut-offs, and thresholds are deliberately left as `<<PLACEHOLDER>>` values for DJ and Vishal to set — the agent ships the *machinery*, never a guessed number. Illustrative (not yet ratified) examples from the framework: pricing-page view ≈ +25 points, demo request ≈ +40, email click ≈ +3, email open ≈ 0, fit grade A floor ≈ 80.

7. Negative scoring and disqualification (with reason codes)

What it does: Subtracts points or removes leads that don't belong — but conservatively, so it doesn't silently kill genuine buyers. Soft-negatives (e.g. hard bounce, spam complaint, junior-only title, free webmail on a B2B form) subtract points. **Hard-disqualifies** exit the lead entirely, each tagged with a **closed-set reason code**.

Example you'd type: *"Disqualify the out-of-ICP rows and tell me why each one was dropped."*

What it produces: A reason-coded list. The reason vocabulary is fixed and shared with sales: `DQ_GEO` (wrong geography), `DQ_COMPETITOR`, `DQ_SUBSCALE` (too small), `DQ_DNC` (opted out), `DQ_CUSTOMER` (already ours), `DQ_UNVERIFIED` (bad email), plus recycle codes like `RC_TIMING`, `RC_NO_BUDGET`, `RC_WRONG_PERSON`. Hard-DQ batches are meant to get a weekly human review to catch false positives.

8. Buying-group / committee mapping

What it does: For a target account, lays out the buying committee and tags each contact with exactly one role: **Economic Buyer** (budget + final yes/no), **Champion** (sells internally), **Influencer**, **End-User**, **Gatekeeper/Blocker**. Each role carries a roll-up weight, so the Economic Buyer's engagement counts most and a blocker's can count negative.

Example you'd type: *"Map the buying group for this target account and tell me who's missing."*

What it produces: A role-labelled committee map plus a **coverage** read-out — e.g. "single-threaded: only a Champion is engaged, Economic Buyer is dark → capped below MQL." It flags single-threaded accounts and recommends a multi-threading play to reach the budget-holder.

9. Tiering and speed-to-lead routing

What it does: Ranks accounts into a "work-now" priority queue by blending fit, intent, committee coverage, and reachability, then sets a response-time clock per tier.

Example you'd type: *"Tier today's MQLs and give me the P1 work-now queue."*

What it produces: A tiered list — **P1 (1:1, hot, ~5-min response target), P2 (1:few), P3 (1:many nurture), Hold/Recycle** — each with a speed-to-lead SLA (the response-time placeholders are founder-set). Lead-to-account matching always runs *first*, so a strategic-account lead routes to its account owner instead of a random rep.

10. Drafting 1-to-1 outreach copy

What it does: Writes personalised, one-to-one email and LinkedIn drafts grounded in citable facts — never spray-and-pray. (This is targeted conversion copy; broadcast posting belongs to the Growth Hacker.)

Example you'd type: *"Draft the outreach for these five A-fit accounts using the Agentic Commerce angle."*

What it produces: Per-row draft copy in markdown — touch 1 is a relevance hook + soft CTA (~75–100 words), follow-ups add new value, the final touch is a permission-to-close "break-up." Source material can be **any doc, PDF, research file, or online source**, not just a Woodpecker template. These are **drafts only** — the first real send is human-gated, and Vishal (the human) refines copy before anything is built. The copy step is explicitly the biggest bottleneck and stays a hard human gate.

11. Building multi-touch nurture sequences (drafts)

What it does: For prospects who aren't ready yet, designs multi-step, behaviour-triggered nurture tracks per persona and funnel stage, re-scoring as engagement builds and recycling "right-fit-not-now" leads (status `RECYCLED`, reason-coded) so they need a fresh trigger — not just the passage of time — to be re-promoted.

Example you'd type: *"Build the nurture sequence for B-fit catalog-ops leads."*

What it produces: A sequence spec (markdown) plus planned CRM enrollments. A core discipline: it reads results as **CTOR over CTR** (click-to-open rate, not raw click rate) and **always separates new audiences from retargeting** before judging any email result, because reading them together is misleading.

12. ABM (account-based marketing) orchestration

What it does: For a set of named target accounts, reads account-level intent (opens, clicks, CTOR) and lays out a coordinated, role-tailored set of touches across the committee — with new-vs-existing-client routing already applied.

Example you'd type: *"Run the ABM play against these ten target accounts."*

What it produces: An ABM orchestration plan (markdown / xlsx): per-account intent read + a coordinated touch plan, routing new accounts to Sales and existing accounts to Key Account Management.

13. Building Woodpecker campaigns — CREATE-ONLY, never send

What it does: This is the headline execution capability, and it is wrapped in heavy guardrails because **Woodpecker (the cold-email engine) is a LIVE production account** — verified 19 Jun 2026 to hold **97 campaigns (3 actively sending, 4 paused, 8 draft, 82 completed)** run by real people. The agent is a **guest**. It can only **create brand-new campaigns** prefixed `[LG-AGENT]`, and it records every campaign ID it creates in a **ledger** (an append-only log — today a local file, with a Zoho custom record as the preferred durable home). "A campaign it owns" = an ID in that ledger, *never* a name. Everything else on the account is strictly read-only.

Example you'd type: *"Stage the campaign / build the Woodpecker sequence for the cleared US segment."*

What it produces (in normal operation): New `[LG-AGENT]` linear campaigns built to Vishal's spec, left in a **non-sending DRAFT/PAUSED state**, ledgered, and announced with a one-line Slack summary (counts and metadata only — never prospect emails). What it will **never** do, enforced in code (`tools/leadgen/wp_client.py`):

- Never call `/run`, `/resume`, `/start`, `/pause`, `/stop`, or `/delete` — on ANY campaign, **including one it just created**. The first send is always a human action. (These verbs literally don't exist as methods; calling one raises `ForbiddenAction`.)
- Never touch a campaign not in its ledger.
- Never reuse a live or primary-domain mailbox. It may only use an **isolated, warmed secondary-domain** mailbox from an allow-list (env `WOODPECKER_ALLOWED_MAILBOX_IDS`). That allow-list is **empty today** — so, by design, the agent currently can't build at all (Woodpecker won't create a campaign with no sender mailbox).
- Never write to a campaign by name-matching — only to the exact ID its own create call returned.

It also respects the **shared, account-wide rate limit**: it serializes its calls (one at a time), backs off exponentially on an HTTP 429, and aborts rather than "retry-storming" and starving Vishal's live campaigns.

14. The pre-send / pre-enroll wall (the four checks)

What it does: Before *any* outbound — and before *enrolling* anyone into Woodpecker (because enrolling writes a person's PII into a third-party store, which counts as a send-side action) — it runs four non-negotiable checks **per record, against fresh live data, in this order**:

1. **Lawful basis** present (Zoho `Data_Processing_Basis` {Legitimate Interests, Contract, Consent-Obtained}).
2. **Synchronous suppression scrub** — block opt-outs/DNC (`Leads.Email_Opt_Out` / `Contacts.Email_Opt_out1`), competitors, and **any existing client or open opportunity** (a multi-field OR-union across `Customer_Type`, `Contact_Type1`, `Customer_Classification`, linked Account/Deal type).
3. **Geo-gate** re-checked (person-level US-only; Europe never cold-emailed).
4. **First-send human approval** token present.

Example you'd type: *"Dry-run the pre-send gate over this cohort before we build anything."* — which runs `python -m leadgen.preflight --prospects leads.csv --template body.txt --subject subj.txt --source clean_sheet --mailbox <warmed_id> --approved <token>`.

What it produces: A structured **ALLOW / HALT** verdict with a reason histogram (row indices + reasons only, never emails). Today the verdict is **HALT**, because 100% of Zoho leads have a blank lawful basis → zero eligible records. The toolkit (`presend_gate.evaluate()`) also fails *closed* on anything ambiguous: a garbled suppression flag counts as suppressed, an unknown region is rejected, and raw Zoho is refused outright as a send source (you must feed a clean Google Sheet, never a raw 40,000-row CRM dump). There's a global kill-switch too: env `WOODPECKER_AGENT_BUILD` must equal `on`, or the agent makes **zero** write calls.

15. Merge-field leak check (the {{token}} guard)

What it does: Before a campaign goes out, renders the email template against every prospect row and catches any personalization token (like `{{company}}` or `{{first_name}}`) that would render literally — the nightmare where a broken merge field ships to all 1,000 recipients.

Example you'd type: *"Run a merge-preview on this template against the list."*

What it produces: A clean/not-clean report (`merge_preview.validate`) naming exactly which rows and which fields would leak. A token with a fallback (`{{first_name|there}}`) is fine; a token with no value and no fallback is flagged. A campaign can't pass the wall unless the merge preview is clean.

16. Enrolling cleared leads into a non-sending campaign

What it does: Adds *only* the records that passed all four checks into an *owned*, *confirmed-DRAFT/PAUSED* campaign. Enrolling into a RUNNING campaign would itself be sending, so it re-reads the campaign status immediately before every enroll and refuses if it's RUNNING.

Example you'd type: *"Enroll the cleared segment into the staged campaign."*

What it produces: Today, **zero** enrollments — because zero records clear lawful basis. The `add_prospects` method re-verifies every record at enroll time and refuses the whole batch on any failure; and even when records would clear, live enroll is intentionally left disabled in this build (stage-zero). The one write it's permitted to do on a prior-run campaign is to **remove** a prospect who has since opted out — never to add one.

17. Reply handling and inbound triage

What it does: Classifies replies and reacts. Stop-on-reply is automatic in Woodpecker; the agent pauses a prospect on out-of-office and stops on "don't email me."

Example you'd type: *"Triage the replies from this campaign."*

What it produces: Each reply tagged from a fixed vocabulary — `INTERESTED`, `NOT_NOW`, `REFERRAL`, `OBJECTION`, `UNSUBSCRIBE`, `OOO` (out-of-office), `WRONG_CONTACT`, `HARD_NO` — with the matching CRM status update (an unsubscribe → suppression; an interested reply → toward SQL).

18. Documenting the multi-campaign Router (branching logic)

What it does: Woodpecker only allows **one** branching condition per campaign, so rich "if they click, do X" logic is handled by an always-on **Router** (a separate piece of infrastructure, outside this agent). The agent doesn't run the Router — it **documents the rules and supplies its ledger** so the Router only acts on the agent's own campaigns.

Example you'd type: *"Document the Router rules for the C0 → C1 sequence set."*

What it produces: A set of small linear "building-block" campaigns (C0 Cold Intro, C1-Hot for clickers, C1-Value for non-engagers, C2-Breakup, C-Nurture) plus a webhook-to-action table (e.g. `link_clicked` → enroll in C1-Hot + Slack hot-lead alert; `prospect_replied` → stop + AE accepts the SQL). The unbreakable rule: **branch on clicks and replies only, NEVER opens** (opens are ~50% fake), and the Router must filter every event by campaign ID against the agent's ledger so it never touches Vishal's live prospects.

19. Booking the meeting and routing the SQL by account status

What it does: Packages a qualified lead into an SQL (Sales-Qualified Lead) context pack and routes it to the right human — **never random round-robin**. Lead-to-account match runs first.

Example you'd type: *"Book the meeting and hand off the SQL."*

What it produces: An accepted-SQL hand-off: a `qualified-pipeline` record in the Spine plus a brief (.md/.docx) containing the committee map, the fit "why", the intent signals, and the "why now." Routing is **by account status** — a brand-new/target account goes to **Sales / New Client Acquisition**; an existing client goes to **Key Account Management as an expansion signal, never a cold pitch**. The agent **cannot self-certify an SQL** — the receiving human accepts it as real pipeline (a deliberate human gate).

20. Open-rate diff and reporting

What it does: Solves a real manual chore Vishal does today: pulling "who opened" — a count that keeps shifting as late opens trickle in days later — and figuring out who is *newly* opened since the last pull.

Example you'd type: *"Diff this open-rate export against last week's file and give me the new openers."*

What it produces: A clean "newly opened since last pull" list to hand to callers — a read-only, low-risk win. (Note: tracking is often deliberately turned OFF in Woodpecker because it hurts deliverability, so this applies where tracking is on.)

21. Pulling call reports from the dialer

What it does: The inside-sales dialer can't be fully integrated, and the dialer *tool* can't operate in India (TRAI rules — so India calls are made manually). What the agent *can* do is pull the call reports daily/weekly and fold them into the picture (e.g. as an "already-contacted" suppression signal).

Example you'd type: *"Pull this week's call reports and flag anyone already being worked."*

What it produces: A report-pull that feeds suppression and routing — so the agent doesn't email someone a rep is already calling.

22. Feeding the Brain with funnel aggregates (no PII)

What it does: Writes the funnel numbers up to the **Brain** (the append-only `_brain/` knowledge folder) so the organisation learns — but **only aggregate numbers, never a single prospect's data**.

Example you'd type: *"Write today's funnel aggregates to the Brain."*

What it produces: An append-only, namespaced feed file `performance-analytics-leadgen-YYMMDD` containing MQL→SQL→meeting conversion broken down by channel, account, and audience type. It dumps the raw pull to `_brain/_raw/` first (raw-first), and never publishes a number it can't trace back to a source. Per-prospect data stays in Zoho; **no PII ever enters the Brain, Slack, or Drive.**

23. Proposing scoring-config changes and surfacing drift

What it does: Watches the funnel for problems (e.g. MQL→SAL acceptance dropping below a floor) and proposes recalibrations — but as a **versioned, human-approved** change, never a silent tweak.

Example you'd type: *"Acceptance is dropping — propose a scoring-config change."*

What it produces: A versioned scoring-config object (.md/.json) with before/after evidence, handed to DJ/Vishal for sign-off. Lowering an engagement floor without raising the fit floor is forbidden by policy.

24. The audit trail (who ran what)

What it does: Logs every run for accountability — directly answering Vishal's question, "if the AI sends wrong at scale, who do you catch?"

Example: This happens automatically on every run (`tools/leadgen/audit.py`).

What it produces: An append-only "who ran what" log (metadata only, no PII), so every campaign creation and every gate decision is traceable.

A note on what's switched OFF today (so this list is honest)

Everything above is **built and safety-verified** (24 safety tests, adversarially checked sound), but the live sending path is intentionally dark:

- **Send-mode is OFF.** The agent never starts a campaign; a human triggers the first send.
- **Zero prospects can be staged today**, because no lead has a lawful basis recorded in Zoho. DJ must sign the LIA (Legitimate Interest Assessment) and a warmed sending mailbox must be provisioned first.
- **The mailbox allow-list is empty**, so the create step itself currently aborts (no sender-less create exists).
- **Live enroll, the Zoho dedup query, the already-contacted feed, and the Router** are deliberately left to follow-up work / other infrastructure.

So if you run it today and ask it to send, the correct, designed answer is a clear **HALT** with the reasons spelled out — not a quiet failure. That caution is the feature, not a bug: brand sanctity beats volume.

Source files for this section (all absolute paths):

- iksula-marketplace/plugins/iksula-agents/skills/lead-gen/SKILL.md
- iksula-marketplace/plugins/iksula-agents/skills/lead-gen/references/sending-stack.md
- iksula-marketplace/plugins/iksula-agents/skills/lead-gen/references/seam-and-compliance.md
- iksula-marketplace/plugins/iksula-agents/skills/lead-gen/references/scoring-framework.md
- iksula-marketplace/plugins/iksula-agents/skills/lead-gen/references/vocabularies.md
- iksula-marketplace/plugins/iksula-agents/tools/leadgen/ (README.md, config.py, presend_gate.py, wp_client.py, merge_preview.py, preflight.py)
- the internal Vishal walkthrough analysis Lead-Gen Workflow - Tools, Tasks & Automation Map - 260619.md

4 The tools and integrations it is connected to

Think of the Lead Gen agent as a careful new hire who is allowed to *use* the company's existing software, but who has been given a very short, locked-down set of keys. It plugs into outside apps to find prospects, build email campaigns, and record what it did — but every dangerous action (actually pressing "send") is physically removed from its keyring.

Below is every external tool, app, and connector it touches, plus its own safety programs. First, two quick definitions you'll see throughout:

- **API (Application Programming Interface):** a software "doorway" that lets one program talk to another directly, without a human clicking buttons.
- **MCP server (Model Context Protocol server):** a standard plug-in adapter that lets the AI talk to an outside app (like Zoho or Woodpecker) in a controlled way. Think of it as a certified power adapter rather than splicing the wires by hand.

The full integration map

Tool / app	What it is	What the agent uses it for	How it connects	LIVE or PENDING today
Apollo	A prospect database (names, work emails, phone numbers, company info)	Sourcing prospect lists. **Filters are set inside Apollo itself**, not in the agent — the agent is poor at filtering big lists	Pulled via the agent's Apollo integration; lists are downloaded as a sheet	LIVE for sourcing, but **credit-limited** (≈45,000 credits per ≈$1,000; email = 1 credit, phone ≈ 8–9, enrichment ≈ 20–25). No safe "unlimited" automation yet
BuiltWith	A "technology lookup" tool (tells you which software a company runs)	Building lists Apollo can't, e.g. "all Adobe partners"	Manual / case-by-case	LIVE but niche, used by hand
LinkedIn / Sales Navigator	The professional network + its paid search tool	Manual company and people discovery for **any region** where Apollo coverage is thin	Manual (no automated connection)	LIVE but **manual only** — hard to automate
Events lists	Speaker and attendee lists from conferences	Sourcing prospects; speaker lists are always available, attendee lists only if Iksula takes a booth	Pulled through the agent in about 5 minutes	Semi-automated
Leadfeeder ("Intent tool")	De-anonymizes website visitors at the **company** level (not the individual person)	Spotting which companies visited the site	—	**De-prioritized / can be dropped** ("not critical") — effectively OFF for v1
GA4 (Google Analytics 4)	Website traffic analytics	Deeper visitor analysis	—	**Not needed now** — not wired in
Zoho CRM	The customer database and **system-of-record** (the single, authoritative, changeable record for every prospect) — US data center, org `29004087`	Reading lawful basis, opt-outs, existing-client flags, scores, lifecycle; and storing the agent's **campaign-creation ledger** (its list of campaigns it built). It is the durable "truth" layer the append-only Brain cannot hold	The **native Zoho MCP server** (`mcp__zoho-crm__…` tools)	LIVE as the record store. **Important caveat:** Zoho holds *raw* data, so it must **never** be used as a send source (see the "40,000-blast trap" below)
Woodpecker	The cold-email **campaign engine** (templates, importing prospects, multi-step sequences, sending, reply handling)	Building new outbound email campaigns to Vishal's spec — **create-only, never auto-send**	The official **Woodpecker MCP server** (`Woodpeckerco/woodpecker-mcp-server`) preferred, or the **v2 REST API** as fallback. Base URL `https://api.woodpecker.co`; auth via header `x-api-key`; key lives only in env var `WOODPECKER_API_KEY`	**LIVE production account** — API verified active 19 Jun 2026. This is the riskiest integration; see the guard rails below
MailerLite	A newsletter / designed-template email tool	Opted-in nurture and broadcast emails to people who already consented (never cold)	—	**Out of scope for v1** — DJ deprioritized it; design-heavy, can't automate. Never put a cold prospect here

Tool / app	What it is	What the agent uses it for	How it connects	LIVE or PENDING today
Dialer / calling tool (exact name not yet identified)	Inside-sales phone-calling software	Pulling daily/weekly **call reports** only (so the agent knows who's already been contacted)	Report-pull only; no live integration	PENDING — only report-pull is feasible. **The dialer cannot be used in India (TRAI rules)**, so India calls are made by hand
Slack	Team chat	Posting one short run-summary (campaign name, id, mailbox, count, run id — **never prospect emails/PII**), hot-lead alerts, and asking a human to approve the first send / accept an SQL	Slack channel messages	LIVE as the human-approval and alert channel
Google Drive (`_brain/` and `_spine/`)	The shared knowledge folders: **Brain** = append-only knowledge + aggregate numbers, no personal data; **Spine** = the shared queues and coordination records	Reading `icp-audience`, `performance-analytics`, `competitor-radar` from the Brain; reading the `content-sourced-lead` queue and channel inputs from the Spine; writing funnel aggregates to `performance-analytics-leadgen-YYMMDD`	The **Google Drive connector** (`@iksula.com` account), via the plugin's `brain_io` helper (`brain_io get` / `brain_io write`)	LIVE — but read Brain files with `download_file_content`, never `read_file_content` (the latter corrupts the CSV)

The agent's own toolkit (the safety programs it must call)

The agent does not hand-roll its own connection logic. It calls a small Python package, `tools/leadgen/`, whose entire job is to make the safety rules **structural** — meaning the dangerous action is *physically absent from the code*, not just discouraged. "Nothing here starts a send. There is no `run`/`start`/`resume`/`pause`/`stop`/`delete` method anywhere."

Program	What it does	Sends or writes personal data?
`config.py`	Single source of truth for the hard constants: the secret env var names, the kill-switch, the mailbox allow-list, and the **only two web requests the agent may ever make** (`GET /rest/v1/campaign_list` to read, `POST /rest/v2/campaigns` to create)	No
`merge_preview.py`	Catches `{{token}}` leaks — e.g. a broken `{{company}}` field that would otherwise print literally to 1,000 recipients — *before* anything is built	No (runs locally)
`presend_gate.py`	**The wall.** Decides ALLOW or HALT by re-checking, per record: lawful basis, suppression scrub, geo-gate, clean-sheet source, warmed mailbox, clean merge-preview, and human approval	No — it only decides
`ledger.py`	The append-only **creation ledger** — the *only* definition of a campaign the agent "owns" (ownership = the campaign id is in this ledger, never the name)	Metadata only
`audit.py`	The append-only "who ran what" log, for accountability	Metadata only
`wp_client.py`	The Woodpecker client. **Read + create-only**, dry-run by default. The web layer is allow-listed to exactly list + create; any other request raises `ForbiddenAction`	Create-only, and only when fully gated

Program	What it does	Sends or writes personal data?
`preflight.py`	Lets an operator dry-run the gate over a list and see PASS/HALT, without creating or sending anything. Run it with `python -m leadgen.preflight …`	No

The three safety facts you should know about the LIVE accounts

1. **Woodpecker is a real, busy account with other people's work in it.** As of 19 Jun 2026 it held **97 campaigns** (3 actively sending, 4 paused, 8 draft, 82 completed), run by several humans, sending from real mailboxes. The agent is a "guest": it only creates new campaigns named with the `[LG-AGENT]` prefix, records every id it makes in the Zoho ledger, and **never pauses, runs, resumes, stops, edits, deletes, or enrolls into any campaign it doesn't own.** The rate limit is shared with those live campaigns, so the agent makes one request at a time and backs off on a "too many requests" (429) error rather than hammering.
2. **Send-mode is OFF, and "create" is not "send."** Building a campaign and adding prospects is *staging*, not sending. The agent **never calls the run/start verb on any campaign — including one it just made.** The first send is always triggered by a human, in Slack, and only after a human approves it.
3. **The correct number of emails today is ZERO, by design.** Enrolling a prospect into Woodpecker writes their personal data into a third-party store, so it triggers the full pre-send wall on every record: lawful basis (the Zoho field `Data_Processing_Basis` must be one of *Legitimate Interests*, *Contract*, or *Consent – Obtained*), a suppression scrub, a US/India-only geo check, a business-email check, a warmed isolated mailbox, a clean merge-preview, and human approval. Today **100% of Zoho leads have a blank** `Data_Processing_Basis`, and **no warmed secondary-domain mailbox has been provisioned** (`WOODPECKER_ALLOWED_MAILBOX_IDS` is empty), so the gate correctly returns **HALT** and zero prospects are staged. There is also a master kill-switch: unless env `WOODPECKER_AGENT_BUILD=on`, the agent makes zero write calls at all.

A related guard worth naming: the **"40,000-blast trap."** If someone with only Zoho access said "email the US clients," a naïve tool could pull tens of thousands of raw CRM addresses and blast them — hitting existing clients, colleagues' live deals, and even Iksula's own competitors. So the wall refuses any send whose source is raw Zoho; the only permitted source is a **clean Google Sheet** that has already been cleaned and de-duplicated (with human permission required before any rows are removed).

5 How to set it up — step by step

This is a beginner-proof checklist. Follow it in order. By the end you'll have the Lead Gen agent installed, connected to the company's shared knowledge and records, and proven safe with a dry run that (correctly) refuses to send anything today.

A quick word on what "safe" means here. The Lead Gen agent is wired so it literally *cannot* send a cold email today. That's on purpose. Several big switches are OFF, and they stay OFF until a human turns them on. So if your dry run says "HALT" at the end, that is the system working exactly as designed — not a setup mistake.

Step 1 — Get the accounts you need (and know why each one matters)

You (or your operator) need logins for these. Think of them as the "utilities" the agent plugs into.

Account	What it's for	Why the agent needs it
@iksula.com Google Workspace (e.g. `claudecode-4@iksula.com`)	Signs you into Google Drive, where the shared "Brain" and "Spine" folders live	The Brain (`_brain/` — append-only shared knowledge and aggregate numbers, never personal data) and Spine (`_spine/` — the shared to-do queues) are Google Drive folders. The connector only sees them when signed in as an @iksula.com account.
Zoho CRM (the US data-center org, id `29004087`)	The "system of record" — the single trusted store for each prospect's record: their lifecycle stage, scores, suppression flags, and the all-important *lawful basis* (recorded permission to email them)	Every safety check reads live from Zoho. No Zoho, no way to confirm a person is OK to email.

Account	What it's for	Why the agent needs it
Woodpecker	The cold-email campaign engine. This is a **LIVE production account** — 97 real campaigns, 3 actively sending	The agent builds new draft campaigns here (create-only). Because real revenue work is running, the agent is treated as a guest.
Slack	Where the agent posts its run summary and where a human gives approvals	Approvals and "hot lead" alerts happen here. Today's approvals are a human replying, not a button.
Apollo (plus BuiltWith / LinkedIn Sales Navigator as fallbacks)	Sourcing prospect lists — emails, filtered by job title and industry	This is where raw lists come from. **Filters are applied inside the Apollo tool, not in Claude** (Claude filters large lists poorly). Apollo is credit-metered, so budget it carefully.

Two things that are **out of scope** so you don't waste time setting them up: **MailerLite** (the monthly newsletter — design-heavy, deprioritized by DJ) and **Leadfeeder / the "Intent" visitor tool** (judged "not critical, drop it" for v1).

Step 2 — Install the plugin (the agent itself)

The Lead Gen agent is one of the "iksula-agents." It comes in two parts that live side by side:

- The **skill** — plain-English instructions the AI follows: `skills/lead-gen/SKILL.md` plus its reference files.
- The **toolkit** — Python code that *enforces* the safety rules so the agent can't break them even if it tried: the `tools/leadgen/` package.

Install the `iksula-agents` plugin so both are available in Claude Code. Once installed, you trigger the agent by typing a plain-English phrase, for example: "run lead gen", "build the Woodpecker sequence", or "score this lead."

Step 3 — Connect and verify the Brain/Drive with `pranaam`

Before doing any Lead Gen work, run the `pranaam` startup skill. (Pranaam is a respectful greeting — fitting, since it's how each session "checks in.") It does two jobs:

1. **Connects** the Google Drive connector using your @iksula.com login.
2. **Verifies** the Brain is actually reachable — that the agent can find and read the `_brain/` folder at runtime.

This matters because of a quirk of the Drive connector: searching for a folder literally named `_brain` comes back **empty**. So the agent instead searches for a known seed file (`brain_io-howto`), takes that file's `parentId`, and *that* is the real `_brain/` folder. The `pranaam` skill handles this for you. If `pranaam` can't verify the Brain, stop — Lead Gen is not allowed to run until the Brain is reachable.

Step 4 — Connect Zoho the safe way (never a shared password)

Zoho is the system of record, so the connection must be clean and auditable. There are exactly two approved ways to connect:

- The **native Zoho MCP server** (the preferred route — MCP is just a standard plug that lets the agent call Zoho's tools), or
- An **OAuth Self Client** (OAuth is a "valet key" sign-in: it grants limited access without ever sharing your actual login).

Never connect by typing a shared username and password into the agent or into chat. That is the one forbidden method. A shared password can't be traced to a person and can't be revoked cleanly — both deal-breakers for a system that touches real customer data.

Once connected, the agent reads specific Zoho fields by their exact internal names (it never guesses from the on-screen labels). The most important is `Data_Processing_Basis` — the lawful-basis field. A send is only permitted when that field holds one of: `Legitimate Interests`, `Contract`, or `Consent - Obtained`. Today **100% of leads have this field blank**, which is why nothing can be sent yet.

Step 5 — Put secrets in local files or environment variables — never in chat

A "secret" here means an API key or token. The hard rule: **secrets live only in a gitignored local file or in an environment variable — never pasted into chat, Slack, or WhatsApp, and never committed to code.** (Gitignored = a file that the version-control system is told to ignore, so it never gets uploaded or shared.)

The toolkit reads these exact names from `config.py`. Set them up by name:

Name (environment variable)	What it holds	Default if you don't set it
`WOODPECKER_API_KEY`	The Woodpecker API key. Can also be sourced from a gitignored local file named `_woodpecker.local`. The key is sent to Woodpecker in a header called `x-api-key`.	Empty — the agent can read campaign metadata only via the transport stub; live calls fail cleanly with "WOODPECKER_API_KEY not set."
`WOODPECKER_AGENT_BUILD`	The master **kill switch**. Must be the exact text `on` to permit *any* write to Woodpecker. Anything else = zero write calls.	`off` — so out of the box the agent makes zero writes.
`WOODPECKER_ALLOWED_MAILBOX_IDS`	A comma-separated list of the *warmed, isolated, secondary-domain* mailbox IDs the agent is allowed to send from	**Empty** — and empty means the agent literally cannot build a campaign at all (a Woodpecker campaign can't be created without at least one sender mailbox). This is correct today: no dedicated warmed cold-send mailbox has been provisioned yet.

A note on that last one: the agent is forbidden from ever using the primary domain `iksula.com` or any real human's mailbox (for example `sam@iksula.com`, `vishal.s@iksula.com`, `vishal.sobti@iksula.com`, `pavan.k@iksula.com`, `subhasan.d@iksula.com`). Cold email must go from a separate, warmed-up mailbox so it can never harm the company's main email reputation.

Step 6 — Know which switches are OFF by default

So there are no surprises, here is everything that is deliberately switched off until a human deliberately turns it on:

- **Send mode is OFF.** The agent never presses "send" — it never calls the `run` command on any campaign, not even one it just created. There is literally no `run`/`start`/`pause`/`stop`/`delete` button in the code; trying to use one raises a "ForbiddenAction" error. The very first send of any sequence is always a human action.
- **The build kill switch (`WOODPECKER_AGENT_BUILD`) is OFF**, so the agent makes zero changes to Woodpecker.
- **No warmed mailbox is allow-listed** (`WOODPECKER_ALLOWED_MAILBOX_IDS` is empty), so no campaign can even be created.
- **Lawful basis is set on zero leads**, so even if everything else were on, the correct number of people to email today is **zero**.
- **Live enroll is disabled** in this build — adding real prospects into Woodpecker refuses with "live enroll is not enabled (stage-zero today)."

The agent's publishing cousin, the Growth Hacker, is similarly OFF by default. None of this is broken — it's the "start fully gated, earn trust later" design.

Step 7 — Verify everything with the safety tests and the dry-run preflight

Now prove the whole thing works *and* is safely locked. Run these from the `tools/` folder (the folder that contains the `leadgen/` package).

First, run the safety tests. These need no network and never touch the live account:

```
python -m unittest leadgen.tests.test_safety
```

Then run the **preflight** — a dry run that asks "would a send be allowed right now?" It loads a prospect list and an email template, runs the merge-preview (the check that catches a broken `{{company}}` field before it renders literally to 1,000 people), runs the full pre-send gate, and prints exactly why a send is held. It **never creates or sends anything**:

```
python -m leadgen.preflight --prospects leads.csv --template body.txt \
    --subject subj.txt --source clean_sheet --mailbox <warmed_id> --approved <token>
```

What you should expect to see today is a verdict of `HALT`, with the reasons listed. The gate only says `ALLOW` when *all* of these hold:

1. the kill switch is on (`WOODPECKER_AGENT_BUILD=on`),
2. the list came from a **clean Google Sheet, never raw Zoho** (pulling 40,000 raw CRM addresses and blasting them is the exact trap this blocks),

3. the sending mailbox is **allow-listed and warmed** (none exists yet),
4. the merge-preview is clean (no leaking `{{tokens}}`),
5. a human **approval token** is present, and
6. **at least one record** passes the lawful-basis + suppression + geo + business-domain checks.

Because every lead's `Data_Processing_Basis` is blank, check #6 comes out to **zero eligible records**, so the verdict is `HALT`. The preflight exits with code `2` on HALT (and `0` only on ALLOW), so an automation wrapper can safely gate on it. Seeing HALT here is your proof that the wall is closed and the setup is correct.

What this gives you

After these seven steps you have: the agent installed; the Brain/Drive connected and verified via `pranaam`; Zoho connected the safe way (MCP or OAuth, never a shared password); secrets stored only in environment variables or gitignored local files; and a verified safety wall that refuses to send until a human warms a mailbox, sets lawful basis on real records, and approves the first send. Until those human steps happen, the Lead Gen agent will build draft campaigns at most — and today, not even that.

Source files:

- `iksula-marketplace/plugins/iksula-agents/skills/lead-gen/SKILL.md`
- `iksula-marketplace/plugins/iksula-agents/skills/lead-gen/references/sending-stack.md`
- `iksula-marketplace/plugins/iksula-agents/skills/lead-gen/references/seam-and-compliance.md`
- `iksula-marketplace/plugins/iksula-agents/tools/leadgen/config.py`
- `iksula-marketplace/plugins/iksula-agents/tools/leadgen/presend_gate.py`
- `iksula-marketplace/plugins/iksula-agents/tools/leadgen/wp_client.py`
- `iksula-marketplace/plugins/iksula-agents/tools/leadgen/preflight.py`
- `iksula-marketplace/plugins/iksula-agents/tools/leadgen/merge_preview.py`
- `iksula-marketplace/plugins/iksula-agents/tools/leadgen/README.md`
- `the internal Vishal walkthrough analysis Lead-Gen Workflow - Tools, Tasks & Automation Map - 260619.md`

6 How to run it — step by step, with a full worked example

This section is the hands-on part: what you actually type, what the agent does at each stage, how to run a safe "would this be allowed?" test, and one full run narrated start to finish. No coding needed.

The trigger phrases you type

You "run" the Lead Gen agent by typing a plain-English trigger into Claude Code (the chat window you talk to the agent in). Any of these wake it up and tell it what job you want:

- **"run lead gen"** — the general start; the agent walks the whole funnel.
- **"work the content-sourced leads"** — process the interest the Growth Hacker agent handed over.
- **"qualify these leads"** / **"score this lead"** — just the scoring step.
- **"de-anonymize the visitors"** — turn anonymous website traffic into named companies/people (geo-gated — explained below).
- **"build the nurture sequence"** — draft a multi-email follow-up track.
- **"build the Woodpecker sequence"** / **"stage the campaign"** / **"load the leads into Woodpecker"** — create (but never send) email campaigns in Woodpecker (Iksula's email-campaign tool).
- **"run the ABM play"** — coordinate outreach against a named target-account list (ABM = Account-Based Marketing, treating one company as the "market").

- **"book the meeting / hand off the SQL"** — package a sales-ready lead and route it to the right owner. (SQL = Sales-Qualified Lead — a lead a salesperson has agreed is real pipeline.)

Tip: run the **"pranaam"** skill first in the same session. That is the start-up routine that connects the agent to the **Brain** (the shared Google Drive `_brain/` folder of knowledge and aggregate numbers) and checks it can actually read it. Lead Gen expects a "Brain-aware session" before it begins.

What the agent does at each phase (the workflow, in plain steps)

The SKILL file lays out a strict, in-order sequence. The agent shows its work and does not skip ahead:

1. **Intake & prime.** It pulls the reference data it needs: `icp-audience` (who counts as an ideal customer), `performance-analytics` (benchmark conversion rates), and `competitor-radar` (competitor context) from the Brain; plus the target-account list and the inbound lead feeds from the **Spine** (the shared Drive `_spine/` folder of queues and to-do records). If anything is missing, it stops and asks — it never makes data up.
2. **Capture & resolve identity.** For each incoming interest event it de-duplicates into one clean record per person, using the email address as the unique key (lowercased, trimmed). It writes an acknowledgement back so the same lead is never processed twice.
3. **De-anonymize & enrich — GATE.** It tries to turn anonymous visitors into named people/companies, but **geo-gated**: person-level identification is **US-only**; for Europe, India and elsewhere it stays at company-level only. Before any of this, or any outbound, it needs **lawful-basis sign-off** (a recorded, human-approved reason you're allowed to email this person). No lawful basis means it holds — it does not proceed.
4. **Qualify — two axes, one gate.** It scores **Fit** ("are you the right kind of buyer?", graded A/B/C/D) and **Intent/Engagement** ("are you showing buying behaviour?") *separately*, then decides jointly. A grade-D fit never becomes a qualified lead no matter how many emails they clicked. Email *opens* count for almost nothing (modern mail apps fake about half of them). Every score comes with a readable "why".
5. **Nurture (draft) & ABM orchestrate.** For people who aren't ready, it drafts a multi-touch follow-up sequence. All of this is **drafts only** — nothing sends here.
6. **Stage into the sending stack (build NEW, don't send).** When a cleared, permission-checked segment exists, it *creates* new email campaigns in Woodpecker, prefixed `[LG-AGENT]`, and writes down every campaign ID it makes in a ledger in Zoho (the Zoho CRM is the system-of-record for per-person lead data). It is a guest on a **live account with 97 real campaigns** — so it creates only, never edits/pauses/runs anyone else's, and **never presses send on anything, including its own**. Campaigns are left in a non-sending state (DRAFT or PAUSED). Adding a person to a campaign is itself a write of personal data, so the four safety checks re-run for every single record at that moment.
7. **Convert & route — GATE.** It books the meeting, packages the context, and routes by account status: brand-new accounts go to Sales/New Client Acquisition; existing clients go to the Key Account Manager as an expansion signal (never a cold pitch). The agent can't certify a lead as "real" itself — the receiving salesperson accepts it.
8. **Feed the Brain & hand off.** It writes funnel totals (aggregates only, no personal data) to a `performance-analytics-leadgen-YYMMDD` file and confirms the Spine/CRM records were updated.

The safety wall that runs before ANY send or enrol

Before the agent puts anyone into a campaign, six conditions must ALL be true (this is enforced in code — `presend_gate.py` — not just a promise the agent tries to remember):

1. The build kill-switch is on (`WOODPECKER_AGENT_BUILD=on`).
2. The prospect list comes from a **clean Google Sheet**, never raw Zoho. (Pulling raw Zoho would blast ~40,000 unfiltered CRM addresses — the "40k-blast trap".)
3. The sending mailbox is **allow-listed and warmed** (a separate, secondary-domain mailbox — never the primary `iksula.com`, never a real person's mailbox). **None is provisioned yet.**
4. The **merge-preview is clean** — no personalization placeholder like `{{company}}` would render literally to recipients.
5. A **human-approval token** is present.
6. **At least one record** passes lawful basis + suppression scrub + geo-gate + being a real business email (no free webmail like gmail.com, no foreign country domains).

Today, **100% of Zoho leads have a blank lawful-basis field**, so condition 6 is zero, and the answer is always **HALT**. That is correct and intended, not a bug.

How to do a SAFE dry-run with the preflight tool

`preflight` is the operator's "would this be allowed?" button. It **never creates anything and never sends anything** — it loads your prospect list and your draft email, runs the merge-preview and the full gate, and prints exactly why a send would be held or allowed.

You run it from the `tools/` **folder** of the iksula-agents plugin (so that `python -m leadgen...` can find the package). The exact command lines, from `preflight.py` and the toolkit README:

```
# from the tools/ folder:

# 1) run the 24 built-in safety tests (no network, no live account)
python -m unittest leadgen.tests.test_safety

# 2) dry-run the gate over a real cohort (never creates / never sends)
python -m leadgen.preflight --prospects leads.csv --template body.txt \
    --subject subj.txt --source clean_sheet --mailbox <warmed_id> --approved <token>
```

What the flags mean:

Flag	What you give it
`--prospects`	a CSV file of prospect rows (the cleaned list)
`--template`	the email body file, using `{{field}}` placeholders
`--subject`	(optional) the subject-line file
`--source`	`clean_sheet` (the only accepted source) or `raw_zoho` (always rejected)
`--mailbox`	the warmed sending mailbox id (must be allow-listed)
`--approved`	the human-approval token (leave it off = "not approved")

How to read the result. It prints a JSON report and then a one-line verdict:

- `=== VERDICT: ALLOW ===` — every condition passed. The tool exits with code 0. A send *could* be staged.
- `=== VERDICT: HALT ===` followed by `Held for: ...` — at least one condition failed; the reasons are listed in plain words. The tool exits with code 2, so an automated wrapper can stop on it.

Reason codes you'll see include `build_switch_off`, `source_not_clean_sheet`, `no_allowlisted_mailbox (none provisioned — infra gap)`, `merge_preview_not_clean (token leak risk)`, `no_human_approval`, and `zero_eligible_records (no record cleared lawful basis + suppression)`. There is also a `reason_histogram` that tallies *why each prospect failed* — useful for seeing patterns (e.g. "200 rows: no_lawful_basis").

One complete worked example, start to finish

The brief. DJ assigns: "Pitch Agentic Commerce to US retail-tech companies." You, the operator (Vishal), type into Claude Code:

run lead gen — work the content-sourced leads, US retail-tech, then stage a Woodpecker campaign

Phase 1–2 (intake, capture). The agent runs `pranaam`, reads `icp-audience` / `performance-analytics` / `competitor-radar` from the Brain, and pulls the inbound leads from the Spine. Say it finds 50 content-sourced interest events plus a 200-row Apollo list you cleaned into a Google Sheet. It de-duplicates them into one record per person.

Phase 3 (de-anon GATE). It checks each record for a recorded lawful basis in Zoho's `Data_Processing_Basis` field. Every record is blank. It flags this immediately — it can enrich at company level but cannot proceed to person-level or outbound.

Phase 4–5 (qualify, draft). It still scores everyone (fit A/B/C/D plus intent), and drafts a 3-touch Woodpecker sequence — touch 1 a relevance hook plus soft call-to-action, touch 2 a value/case-study angle, touch 3 a permission-to-close break-up. These are **drafts**, with merge fields like `{{first_name}}` and `{{company}}`.

The dry-run. Before staging, you sanity-check it. From the `tools/` folder:

```
python -m leadgen.preflight --prospects retail_tech_us_260623.csv \
    --template c0_body.txt --subject c0_subj.txt --source clean_sheet
```

(Notice: you left off `--mailbox` and `--approved` because neither exists yet.) The report comes back, ending:

```
=== VERDICT: HALT ===
Held for: no_allowlisted_mailbox (none provisioned — infra gap); no_human_approval; zero_eligible_reco
rds (no record cleared lawful basis + suppression)
```

The `reason_histogram` shows all 200 rows failed with `no_lawful_basis` and `region_unknown_or_not_permitted` (the geo-gate fails closed when it can't confirm jurisdiction). The exit code is 2.

Phase 6 (stage). Because the gate is HALT, the agent **stages zero prospects** into Woodpecker. It does not create an empty campaign just to look busy; with nothing cleared, there is nothing to build. The kill-switch (`WOODPECKER_AGENT_BUILD`) is off anyway, so it would make zero write calls regardless.

The outcome today — HOLD. The agent reports back something like:

Processed 200 retail-tech prospects + 50 seam events. Scored and de-duped. Drafted a 3-touch C0→C1→C2 sequence. **Pre-send gate verdict: HALT. Staged 0 prospects into Woodpecker.** Held for: zero lawful basis on any record; no warmed isolated mailbox provisioned; no human approval. Drafts and scores saved; nothing sent, nothing created on the live account.

This is the **correct** result right now. The right number of emails to send today is **zero** until three things change (the pilot path from the sending-stack reference):

1. DJ signs the LIA (Legitimate Interest Assessment) and the team sets `Data_Processing_Basis` on a small, genuinely-consented segment of Zoho records.
2. Infra provisions an allow-listed, warmed secondary-domain mailbox (and its id is added to `WOODPECKER_ALLOWED_MAILBOX_IDS`).
3. A human reviews and provides the approval token, then a **human** triggers the first send — never the agent.

Until then, every honest run of Lead Gen ends exactly like this one: full analysis and clean drafts, and a HOLD at the wall.

7 Guardrails & safety, current limits, and troubleshooting

This is the "what could go wrong, and what stops it" section. The Lead Gen agent works on a **live, real-money email account** and with **real people's personal data**. So it was built the careful way: the dangerous actions aren't just discouraged in the instructions — they're physically blocked in the code (a Python package at `tools/leadgen/`). The agent literally cannot press "send." Below is the plain-English version.

The one-line summary

Today, the correct number of cold emails for this agent to send is **ZERO** — and the system is built so that zero is exactly what happens, on purpose, until three real-world things get fixed (permission-to-email recorded in the CRM, a warmed-up sending mailbox, and a human flipping the switch). Everything the agent does right now is **build-and-stage only** (prepare a campaign and leave it switched off), never **send**.

What it WILL do vs. what it WON'T do

Think of the agent as a careful temp worker on someone else's email account. It will set things up; it will never flip the lights on.

It WILL do	It WON'T (cannot) do
Source prospect lists (from Apollo, BuiltWith, LinkedIn) and clean/de-duplicate them	Auto-delete rows during cleaning — it **highlights** duplicates/existing-clients and removes them only after you say yes
Score leads on two separate axes — Fit (are they the right WHO) and Intent (what they actually DID)	Collapse those into one number, or let a grade-D lead become a qualified lead no matter how many clicks they have

It WILL do	It WON'T (cannot) do
Create **brand-new** Woodpecker campaigns, named with the prefix `[LG-AGENT]`	Touch any of the 97 existing campaigns (3 are running real revenue work) — no pause, run, edit, delete, or enroll-into
Build a campaign in DRAFT/PAUSED (a non-sending state)	Call `/run`, `/start`, `/resume`, or set a campaign to RUNNING — **on any campaign, including one it just made**
Record every campaign id it creates in a "creation ledger" (an append-only log — **today a local file** `tools/leadgen/_state/creation_ledger.jsonl`; a Zoho custom record is the *preferred* durable home)	Decide it "owns" a campaign by its name — ownership is **only** the id written in that ledger
Re-check permission-to-email (lawful basis), suppression, and geography for **every** prospect, every time	Enroll anyone whose CRM record doesn't show recorded permission — today that's everyone, so it enrolls no one
Use only an isolated, warmed-up secondary-domain mailbox you've allow-listed	Borrow a real human's mailbox (e.g. `sam@iksula.com`, `vishal.s@iksula.com`) or the primary `iksula.com` domain
Draft personalised 1-to-1 outreach copy and run a merge-preview to catch broken fields	Let a broken field like `{{company}}` go out as literal text to a thousand people
Send funnel **counts and totals** up to the Brain (the shared knowledge folder)	Put any person's name, email, or PII into the Brain, Drive, or Slack
Hand a qualified meeting (SQL) to Sales or KAM	Self-certify that lead as "real pipeline" — the receiving human accepts it

The kill-switches (and they are OFF by default)

There are three "off switches" — environment settings (named values the program reads at start-up). All three default to OFF/empty, which means "do nothing risky":

- `WOODPECKER_AGENT_BUILD` — the master build switch. Unless it is set exactly to `on`, the agent makes **zero write calls** of any kind (no create, no enroll). Default: `off`.
- `WOODPECKER_ALLOWED_MAILBOX_IDS` — the allow-list of approved, warmed sending mailboxes. **Empty by default**, and an empty list means the agent **cannot build at all** (Woodpecker won't create a campaign without at least one sender mailbox). No mailbox provisioned = no campaign, full stop.
- `WOODPECKER_API_KEY` — the secret key for the Woodpecker account. Lives only in the environment or a git-ignored local file, never in the code. (Referred to by name only; the value is never shown or shared.)

On top of the switches, there are hard **caps**: at most **5 campaigns per run** (`MAX_CAMPAIGNS_PER_RUN`), and the API calls are deliberately slowed to one-at-a-time with back-off, because the rate limit is **shared** with Vishal's live campaigns — the agent must never hog it and starve real sending.

There is also a deeper, "belt-and-suspenders" block: the only two internet requests the code is *allowed* to make are **"list campaigns"** (read) and **"create campaign"** (build). Any other request — a hand-crafted `/run`, a "set status to RUNNING," a delete — is refused before it leaves the building. The Woodpecker helper class simply has **no method** to start, pause, resume, stop, or delete. If anything tries to call one of those verbs, it raises an error called `ForbiddenAction`. So "the agent cannot send" isn't a promise it has to remember — it's missing from the toolbox.

Exactly why the agent HALTs today

When you dry-run the gate (the safety wall) right now, it returns **HALT**, not ALLOW. A send/enroll is allowed **only if ALL six** of these hold — and several fail today:

1. **Build switch on** — `WOODPECKER_AGENT_BUILD=on`. *(Off by default.)*
2. **Source is a clean Google Sheet, never raw Zoho** — this stops the "email the US clients" disaster where the agent would otherwise pull 40,000 raw CRM addresses and blast existing clients and competitors.
3. **Sending mailbox is allow-listed and warmed** — never the primary/live domain. **None has been provisioned yet** (as of 2026-06-19 only two secondary-domain mailboxes exist, both Vishal's; no dedicated warmed cold-send mailbox).
4. **Merge-preview is clean** — no `{{token}}` would render as literal text.
5. **A human approval token is present** — a person explicitly approved this send.

6. **At least one record passes** lawful basis + suppression + geo + business-domain checks.

The decisive one is **#6**. "Lawful basis" (recorded permission to email someone — the Zoho field `Data_Processing_Basis`) is **null on 100% of leads today**. The permitted values are `Legitimate Interests`, `Contract`, or `Consent - Obtained`. With every record blank, **zero** records pass, so #6 fails and the whole cohort verdict is HALT. The README says this plainly: *"With 100% of Zoho leads at null Data_Processing_Basis, #6 = 0 → HALT. That is correct and intended."*

A few more reasons an individual prospect gets dropped (the per-record checks, which **fail closed** — when in doubt, exclude):

- `opted_out` — they're on the unsubscribe/DNC list (`Email_Opt_Out`).
- `existing_client` — they're already a customer or a converted lead; never cold-email a client (route to KAM instead).
- `competitor` / `internal_or_test` — a competitor or an internal/test address.
- `already_contacted_suppress` — a colleague is already working this lead.
- `free_webmail` — a personal gmail/yahoo/outlook address (not a real business contact).
- `foreign_cctld` — the email domain ends in a foreign country code (e.g. `.uk`, `.de`, `.eu`, `.au`, `.in`) that hides a foreign data subject inside a "USA" tag.
- `region_unknown_or_not_permitted` — the region isn't on the cold-allowed list (only US/India), or is simply blank. A blank region is **not** treated as safe; it's excluded, because we can't confirm the jurisdiction.

What is intentionally NOT built yet (and why)

These are deliberate gaps, blocked on work owned by other people, not bugs:

- **Live enrolment (actually writing prospects into Woodpecker)** — switched off. `add_prospects` refuses any live write today and raises *"live enroll is not enabled in this build (stage-zero today)."* It stays off until there's lawful basis to enrol anyone.
- **The Zoho "is this an existing client / already contacted?" lookup** — the wall *uses* these answers as inputs, but the live read happens via the Zoho connector or a reviewed follow-up step; the automated query isn't wired in yet.
- **The "already-contacted" suppression feed** — needs the dialer's call-report export (so the agent knows who Inside Sales has already phoned). Not built.
- **The Router** (the always-on service that moves prospects between campaigns on clicks/replies) — that's infrastructure, outside this agent.
- **Send-mode itself** — off by default everywhere. The first real send is always a **human action**.
- **Leadfeeder/Intent tool and the MailerLite newsletter** — deliberately dropped from version 1. Vishal called Intent "not critical" and the newsletter is design-heavy and out of scope.

Troubleshooting / FAQ

"The agent said HALT / zero eligible — is it broken?"

No. That is the system working correctly. Today HALT is the *right* answer because no lead has recorded permission-to-email. Run the dry-run check to see the exact reasons:

```
python -m leadgen.preflight --prospects leads.csv --template body.txt --subject subj.txt
--source clean_sheet --mailbox <warmed_id> --approved <token>
```

It prints a "Held for:" line listing every reason. It exits 0 only on ALLOW, 2 on HALT.

"I want to test a real send today. How do I force it?"

You can't, and that's intentional. Even if you set the build switch on, you'd still be blocked because there's no warmed allow-listed mailbox and no lead has lawful basis. The safe path is the pilot sequence: (1) Vishal locks a sequence, (2) infra warms a secondary domain and provisions an isolated mailbox, (3) you set `Data_Processing_Basis` on **one small, genuinely-consented segment** in Zoho, (4) the agent builds and stages, (5) a human re-scrubs and approves, (6) **a human** triggers the first send.

"It says 'no_allowlisted_mailbox' or 'no_mailbox'."

There's no warmed, isolated sending mailbox provisioned yet — this is a known infra gap. The agent refuses to use a live or primary `iksula.com` mailbox because those have shared daily send caps and would damage the main domain's reputation. Ask infra to warm a secondary-domain mailbox (3–6 weeks for a new domain) and add its id to `WOODPECKER_ALLOWED_MAILBOX_IDS`.

"It says 'source_not_clean_sheet'."

You pointed it at raw Zoho. The agent will never use the raw CRM as a send source (that's the 40,000-address-blast trap). Export a cleaned, de-duplicated Google Sheet of the specific cohort and pass that with `--source clean_sheet`.

"The merge-preview flagged leaking rows."

Some prospects are missing a field your template uses (e.g. a row has no `company`, so `{{company}}` would print literally). Fix it one of two ways: fill the missing data in the sheet, or add a fallback in the template like `{{first_name|there}}`. The send stays blocked until **no** row leaks **any** field.

"Can the agent fix/refresh a campaign it made last week?"

No. There is no edit or refresh path — create-only. If a campaign for that segment already exists in the ledger, the agent **aborts and asks a human** rather than re-editing it. The single exception: it may **remove** (never add) a prospect from one of its own ledgered campaigns to honour a fresh opt-out.

"I see an `[LG-AGENT]`-named campaign that isn't in the ledger. What happens?"

The agent treats it as **foreign and read-only** and flags it to a human for reconciliation. Ownership is the id in the ledger, never the name — so a stray copy with the right name gets hands-off treatment, not assumed ownership.

"Why won't it just delete duplicate rows during cleaning?"

Per DJ's 2026-06-23 correction, dedup must **highlight** candidates and remove them **only after you give permission** — never auto-delete. The agent could be wrong about who's a client, and a silent deletion is unrecoverable. Brand safety beats speed here: as Vishal put it, missing some volume is fine, but a bad impression damages the brand.

"Is anything actually being sent right now?"

No. The agent is in build-and-stage-only mode. Campaigns are left in DRAFT/PAUSED, send-mode is off, no live enrolment is happening, and the first send is a human action that hasn't been taken yet.

8 Glossary — every term in plain English

This is a plain-English dictionary for everything in the Lead Gen agent's world. Each entry is one sentence. Terms are grouped so related ideas sit together, and within each group they run roughly in the order you meet them.

The big picture — the three parts of the system

- **iKshana** — Iksula's "AI-native" (run mostly by software agents instead of people) marketing-and-sales organisation; a chain of AI agents that hands work down a line from idea to booked meeting.
- **Agent** — a single AI worker with one job (here, "Lead Gen"); you start it by typing a trigger like "run lead gen" in Claude Code (the chat tool you type commands into).
- **Brain** — a shared, read-mostly knowledge store (a Google Drive folder called `_brain/`) that holds facts and **aggregate** (summed-up, no-names) numbers; it is **append-only** (you can add, never overwrite) and **never** holds personal data about a single person.
- **Spine** — the shared "to-do board and message queue" (a Google Drive folder called `_spine/`) where agents drop work for each other and leave coordination records; unlike the Brain, its records can change.
- **Hands** — the agents themselves: the parts that actually do things (source lists, build campaigns, route leads).
- **Zoho CRM** — the official "system-of-record" (the one true, up-to-date file) for per-person lead data — name, email, score, consent, lifecycle; it is the live, changeable store the append-only Brain cannot be (US data centre, org `29004087`).
- **Skill** — the markdown (plain-text-with-formatting) instruction sheet the AI reads and follows to behave like a given agent.
- **Toolkit** — the actual Python code (under `tools/leadgen/`) that **enforces** the safety rules, so the agent literally cannot do the dangerous thing even if it "wanted" to.

- **Seam** — the single, one-way handoff point between two agents; here it is the bridge where the Growth Hacker drops raw interest events and Lead Gen picks them up.
- **content-sourced-lead** — the name of that seam: each record is one "someone showed interest" event handed from the Growth Hacker to Lead Gen.
- **Conductor / Router (always-on)** — a separate piece of automation (outside this agent) that moves prospects between campaigns automatically; until it is live, a human or the agent checks the seam by hand ("polls" it).

The people

- **DJ** — Iksula's founder; sets the vision and signs the legal go-aheads (like the LIA, below).
- **Vishal** — the operator who runs Woodpecker and Zoho day-to-day, designs the email sequences, and owns the email copy.
- **Yatin** — the builder who writes the agents.

The funnel — turning interest into a sales meeting

- **Funnel** — the staged journey a prospect travels: capture interest → qualify → nurture → convert to a booked meeting.
- **Lead** — an engaged person attached to a company, before they have been graded for fit.
- **Prospect** — a potential buyer the agent might contact; a lead the agent is working.
- **Lifecycle stage** — *where* a record is in the funnel; the fixed ladder is `SUBSCRIBER → LEAD → MQL → SAL → SQL → OPPORTUNITY → CUSTOMER`.
- **Lead status** — the *operational state within* a stage (e.g. `NEW`, `WORKING`, `ON_HOLD`, `SUPPRESSED`); a record always has exactly one stage **and** one status, and the two are never mixed up.
- **MQL (Marketing-Qualified Lead)** — a buying group that has passed both the fit gate and the engagement gate; "warm enough for sales to look at."
- **SAL (Sales-Accepted Lead)** — an MQL that an inside-sales rep has **accepted** as worth working.
- **SQL (Sales-Qualified Lead)** — a deeper-qualified lead with a meeting held and real, evidence-backed need confirmed; this is the agent's final output (it can never declare an SQL itself — a human receiver accepts it).
- **Opportunity** — an SQL that a sales rep (AE) has accepted and opened a deal record for; out of Lead Gen's scope.
- **Customer** — a closed-won deal; out of scope, and a reason to **suppress** (never cold-email them).
- **ICP (Ideal Customer Profile)** — the written description of exactly who Iksula wants as a customer (industry, size, geography, tech they use).
- **Firmographic** — company-level facts (industry, size, revenue/GMV band, business model) used to judge fit.
- **Technographic** — facts about the technology a company uses (its e-commerce platform, marketplace presence, software stack).
- **Buying group / buying committee** — the set of people who jointly make a B2B purchase, not one person; the agent scores the **group**, not the loudest clicker.
- **Buyer roles** — the five committee seats each contact is tagged into: **Economic Buyer** (controls budget and the final yes/no), **Champion** (sells you internally), **Influencer** (shapes the requirements), **End-User** (uses the service daily), **Gatekeeper/Blocker** (controls access or resists).
- **BGS (Buying-Group Score)** — the engagement score rolled up across the whole committee, weighted by each person's role.
- **Single-threaded** — an account where only one role is engaged; it is capped below MQL until the committee is better covered, so a lone champion's clicks can't fake a hot lead.

Scoring the lead

- **Fit (the WHO axis)** — how well a company/contact matches the ICP, graded **A / B / C / D**; grade **D never becomes an MQL** no matter how interested they seem.
- **Intent / Engagement (the WHAT-THEY-DO axis)** — how much buying behaviour a contact is showing (page views, downloads, replies), scored separately from fit.

- **Two axes, one gate** — the rule that fit and intent are scored **separately** but a lead must clear **both** to become an MQL; collapsing them into one number is the #1 cause of low-quality lead flooding.
- **Time-decay** — automatically shrinking a lead's score as they go quiet, so old "zombie" leads don't sit forever above the threshold.
- **Negative scoring** — subtracting points for bad signals (bounce, spam complaint, junior title, free-email domain); kept deliberately gentle so it doesn't wrongly bury real buyers.
- **Hard-DQ (disqualify)** — kicking a lead out of the funnel entirely, always with a **reason code** (e.g. `DQ_GEO`, `DQ_COMPETITOR`, `DQ_CUSTOMER`) so the rejection teaches the model instead of dying silently.
- **Recycle** — parking a "right-fit-but-not-now" lead back into nurture (status `RECYCLED`), to be re-promoted only on a fresh signal, never just the passage of time.
- **Tiering** — ranking accounts (P1 1:1, P2 1:few, P3 1:many) so limited human/agent time goes to the best ones first.
- **Speed-to-lead** — how fast you make first contact after a signal; hot inbound leads can be ~9× harder to qualify if not touched within minutes.
- **Placeholder (`<<PLACEHOLDER>>`)** — a deliberately blank value in the scoring rules (e.g. `<<MQL_FIT_FLOOR>>`) that DJ and Vishal must fill in; the agent ships the *machinery* and is forbidden from guessing a number to fill the gap.
- **Buying-group coverage** — the share of required committee roles that have at least one engaged, verified contact; too low means "do not MQL yet."

Sourcing and cleaning the list

- **Apollo** — the main prospect-sourcing tool (filtered lists, emails, phone numbers); filters are applied **inside Apollo**, not in Claude, and it runs on a paid **credit** budget.
- **Credit** — Apollo's pay-as-you-go unit (~45,000 per ~$1,000); an email costs 1, a phone ~8–9, full enrichment ~20–25, so the agent must budget them and never run them dry.
- **BuiltWith** — a sourcing tool for technology-based lists Apollo can't filter (e.g. "all Adobe partners").
- **LinkedIn / Sales Navigator** — manual people-and-company discovery, used as a fallback wherever Apollo coverage is thin (in any region, not just MENA/SEA).
- **Leadfeeder** — a website-visitor de-anonymisation tool (company-level only); explicitly **dropped from v1** as "not critical."
- **Enrichment** — filling in the missing details on a contact/company (title, firmographics, technographics), each field stamped with where it came from.
- **De-anonymise** — turning an anonymous website visitor into a known company or person; **person-level** de-anon is **US-only**, **company-level** for everyone else.
- **Dedup (de-duplication)** — merging records that are the same person/company into one canonical record; here it must **highlight** suspects and delete **only after a human okays it**, never silently.
- **Survivorship** — the rule for which value "wins" when merging duplicate records.
- **Canonical record / canonical key** — the single master record for a person, keyed on their normalised (lowercased, trimmed) email when one exists.
- **Merge field / merge token** — a personalisation placeholder in an email like `{{first_name}}` that gets filled per recipient; if it has no value and no fallback it "leaks" the literal `{{first_name}}` text into the sent email.
- **Merge-preview** — the safety check (`merge_preview.py`) that renders the template against every row first and flags exactly which rows/fields would leak before any send.

Compliance — the legal wall before sending

- **Lawful basis** — the recorded legal reason you are allowed to email a given person (e.g. Legitimate Interests, Contract, Consent-Obtained); stored in Zoho field `Data_Processing_Basis`, and today it is set on **nobody**, so the correct number of emails to send is **zero**.
- **LIA (Legitimate Interest Assessment)** — the written legal justification DJ must sign before "Legitimate Interests" can be used as a lawful basis to email a cohort.
- **GDPR** — Europe's data-protection law; under it Iksula does **not** cold-email Europe at all.

- **DPDP** — India's Digital Personal Data Protection Act; tracked B2B work-emails are in scope, so emailing India needs a lawful basis too.
- **CAN-SPAM** — the US anti-spam law governing commercial email (clear sender, working unsubscribe, etc.).
- **Geo-gate** — the rule that re-checks a person's region before acting: person-level de-anon is **US-IP-only**; EU/India/other is **company-level only**; in the code an unknown region **fails closed** (is blocked).
- **Suppression** — the do-not-contact list; a **suppression scrub** is the live check, run fresh before every send, that blocks opt-outs, DNC, competitors, internal/test addresses, and existing clients/open deals.
- **DNC (Do-Not-Contact) / opt-out** — a person who has said "don't email me"; checked in Zoho via `Leads.Email_Opt_Out` / `Contacts.Email_Opt_out1`.
- **ECS / existing client** — an account Iksula already serves; never cold-pitched and instead routed to KAM as a possible expansion ("ECS" is an overloaded label, so the code matches by field, not by the word).
- **Fail-closed / fail-safe** — when a value is missing or garbled, the code defaults to the **safe** answer (treat as suppressed / region not permitted) rather than letting it slip through.
- **First-send human gate** — the rule that a human must explicitly approve the very first send of any new sequence; the agent drafts and proposes, a person presses go.
- **DSR (Data Subject Request)** — a person's legal request to see/delete their data (under GDPR/DPDP); handled by a human.

The sending stack

- **Woodpecker** — the cold-email campaign engine (sequences, sending, reply-handling); this is a **live production account** with 97 real campaigns, so the agent is treated as a guest.
- **Mailerlite** — the **opted-in** newsletter/broadcast tool; only for people who consented, never for cold prospects, and explicitly out of v1 scope.
- **Create-only** — the agent may **build new** campaigns but may never edit, refresh, run, pause, resume, stop, delete, or touch any campaign it didn't create.
- `[LG-AGENT]` — the name prefix on every campaign the agent builds, for human readability (but ownership is decided by the ledger, not the name).
- **Ledger** — the append-only list (`creation_ledger.jsonl`, ideally in Zoho) of every campaign id the agent created; **"owned" = id is in the ledger**, never inferred from the name.
- **Enroll** — adding prospects to a campaign; this is a **PII write** into a third-party store, so it re-runs all four pre-send checks per record — today that clears zero people.
- **Staging vs sending** — *staging* = building a campaign and enrolling cleared leads while leaving it **non-sending**; *sending* = a human turning it on; the agent only ever stages.
- **DRAFT / PAUSED** — the two **non-sending** states a campaign may be left in; the agent confirms a campaign is in one of these before any enroll.
- `/run` — Woodpecker's "start sending" command; the agent **never** calls it on any campaign, including its own — that verb doesn't even exist in the toolkit.
- **Send-mode OFF** — the default posture: nothing actually emails anyone until a human triggers the first send.
- **Kill-switch** — the master off-switch, the environment variable `WOODPECKER_AGENT_BUILD`; unless it is set to `on`, the agent makes **zero** write calls.
- **PII (Personally Identifiable Information)** — data that identifies a person (name, email); it lives only in Zoho/Woodpecker, **never** in the Brain, Drive, or Slack.
- **Mailbox isolation** — the rule that cold campaigns may only use a warmed, isolated **secondary-domain** mailbox from an allow-list; never the primary `iksula.com` or a real person's mailbox.
- **Warm-up (warming)** — gradually building a new sending domain's reputation over 3–6 weeks so its emails land in inboxes, not spam; not built into Woodpecker, so it's a separate infrastructure need.
- **Daily send cap** — the per-mailbox *technical* limit on emails per day (**~20–49/day**), shared across all campaigns on that mailbox, so "send-mode off" does **not** protect it. (Separately, Vishal manually sets a **60–70/day** *operating* cap on his own sends — a different number, not the same thing.)

- **Deliverability** — whether your emails actually reach the inbox (vs spam/bounce); protected by warming, low volume, clean lists, and turning off tracking.
- **CTR vs CTOR** — **Click-Through Rate** (clicks ÷ emails sent) vs **Click-To-Open Rate** (clicks ÷ opens); the agent prefers **CTOR** as the truer engagement signal.
- **Opens (and why ignored)** — email "open" events are inflated ~50% by Apple Mail Privacy Protection (MPP), so opens are de-weighted to near-zero and branches are based on **clicks/replies only**.
- **Stop-on-reply** — Woodpecker automatically stops emailing a prospect once they reply; the agent never re-sends after a reply.
- **Audience type (new vs retargeting)** — whether an email list is fresh contacts or people re-targeted; results must be read **separately**, because mixing them gives a misleading number.
- **Bounce / spam-complaint rate** — the share of emails that fail (bounce) or get marked spam; deliverability proof requires bounce under 2% and spam under 0.3% before widening.

Connections and tooling jargon

- **MCP (Model Context Protocol)** — a standard way for the AI to call an outside tool/service (here, the Zoho MCP server and the official Woodpecker MCP server); the same safety rules bind whether the agent uses MCP or raw web calls.
- **Connector** — a configured link to an outside service (e.g. the Google Drive connector that lets the agent reach the Brain, the Woodpecker API connector).
- **API key** — the secret password for a service (e.g. Woodpecker's `x-api-key`); referred to by name only and stored in an environment variable (`WOODPECKER_API_KEY`), never written into code, Slack, or files.
- `x-api-key` — the specific HTTP header name Woodpecker uses to receive that secret key.
- **Environment variable (env var)** — a setting read from the operating system at run-time (e.g. `WOODPECKER_AGENT_BUILD`, `WOODPECKER_ALLOWED_MAILBOX_IDS`) rather than hardcoded into the program.
- **HTTP allow-list** — the short, fixed set of web requests the toolkit will ever make (only "list campaigns" and "create campaign"); anything else is refused, making "the agent cannot send" a structural fact.
- **Rate limit / 429** — a cap on how many requests per moment Woodpecker allows (shared with the live campaigns); HTTP error **429** means "too many," and the toolkit backs off rather than hammering.
- **brain_io** — the helper the agent uses to read from and append to the Brain; you call it by its verbs (get/write), never by a hardcoded folder path.
- **brain_io-howto** — the live instruction "seed file" in `_brain/` whose `parentId` (its containing folder's id) is how the agent finds the Brain, because a plain folder-name search returns empty on this connector.
- `download_file_content` **vs** `read_file_content` — the correct vs wrong way to read a Brain file; the wrong one reformats and **corrupts CSV data**, so it is never used.
- **performance-analytics-leadgen-YYMMDD** — the one Brain feed Lead Gen writes: funnel **aggregates only** (counts and conversion rates), namespaced and dated, never per-person rows.
- **Namespaced** — tagging a write (with `-leadgen-`) so each agent owns its own lane and writers never collide.
- **Raw-first** — the discipline of dumping the untouched source data to `_brain/_raw/` before publishing any summary, so every number is traceable.

Identifiers, codes and process terms

- **ULID** — a sortable unique id (Universally Unique Lexicographically-sortable IDentifier) used as the `correlation_id`; think "a timestamped ticket number."
- **correlation_id** — the ULID that uniquely tags one handoff event so re-deliveries are recognised and ignored (a "no-op").
- **Idempotent** — an operation that is safe to repeat: doing it twice has the same effect as once (re-reading a seam event changes nothing).
- **No-op** — "no operation"; an action that deliberately does nothing because it was already done.
- **ACK (acknowledgement)** — the note Lead Gen writes back onto a consumed seam record (`received` / `de-duped` / `rejected` + reason) so the sender knows it was handled.

- **At-least-once delivery** — the assumption that a message may arrive more than once, which is why consuming must be idempotent.
- **lawful_basis_tag** — an advisory region/permission stamp the Growth Hacker puts on a seam event; Lead Gen treats it as a hint but **enforces** the real check itself.
- **HITL (Human-In-The-Loop)** — any step where a human must approve before the agent proceeds (e.g. first send, scoring-config change, SQL acceptance).
- **Voice gate / Slack gate** — the human-approval checkpoint posted to Slack where a person replies "yes" before the agent (or a human) acts.
- **Audit trail / audit log** — the append-only "who ran what, when" record (`audit_log.jsonl`) that makes the agent accountable, since you can't discipline an AI the way you can a person.
- **Preflight** — the dry-run command (`python -m leadgen.preflight`) that runs the whole pre-send gate over a list and prints PASS/HALT **without** creating or sending anything (today it prints HALT).
- **Dry-run** — a simulated execution that makes no real changes or network calls; the Woodpecker client defaults to this.
- **The wall / pre-send gate** — `presend_gate.evaluate()`, the code that returns ALLOW or HALT; a send is allowed only if all six conditions hold (build switch on, source is a clean Sheet, mailbox warmed + allow-listed, merge-preview clean, human approval present, and at least one record passes lawful-basis + suppression + geo + B2B-domain), and today it returns **HALT, zero eligible**.
- **Clean Sheet vs raw Zoho** — the agent may only send from a **clean, human-reviewed Google Sheet**, never from the raw 40,000-row Zoho export (the "blast trap"); the gate refuses a raw-Zoho source.
- `ForbiddenAction` — the error the toolkit throws the instant any banned verb (run/pause/delete/etc.) is attempted, on any campaign including its own.
- **Content-sourced lead** — a lead that originated from the Growth Hacker's content/organic surfaces (a LinkedIn post, blog form, tracked link), arriving via the seam as the canonical "Online" channel entry.
- **Four channels** — Lead Gen's single funnel feeds from four sources: **Online** (the seam), **Events**, **Inside Sales**, and **Reference**; Reference leads arrive pre-qualified and **bypass MQL**.
- **TRAI / DLT / DND** — India's telecom-marketing rules; only the **dialer tool** is India-blocked by them, so India calls are made manually (calling itself is not India-only).
- **KAM (Key Account Management)** — the team that grows existing accounts; existing-client signals route here as expansion, never as a cold pitch.
- **NCA (New Client Acquisition) / Sales / AE / SDR / BDR** — the human sales roles the agent hands SQLs to: Sales/NCA win new logos, an **AE** (Account Executive) owns deals, and **SDR/BDR** (Sales/Business Development Reps) do the first qualification.

9 Fine print, known rough edges & extra answers

Small but important details that did not fit neatly above. Read this once before you operate the agent.

How the four channels actually arrive

Lead Gen serves one funnel fed by four channels, but they do **not** arrive the same way:

- **Online** is the only one that comes through **the seam** (the `content-sourced-lead` queue from the Growth Hacker).
- **Events, Inside Sales, and Reference** arrive as **separate** `channel-input` **records in the Spine** (not the seam).
- **Reference** leads are **pre-qualified** — they skip the MQL test entirely (applying MQL scoring to a referral is a mistake, not a low score).

"Human-gated" is not one blanket rule — autonomy is per channel

Different channels have different default autonomy levels:

- **Cold email** — autonomous *within strict limits* (caps + the pre-send gate), still no first send without approval;
- **LinkedIn** — *approve-before-act* (a human okays each action);

- **Phone** — *human-led* (people make the calls; the agent only supports/report).

So "the human approves" looks different depending on the channel.

Known rough edge: India is allowed, but `.in` emails are currently dropped

The send gate's allowed cold-email regions **include India**. But the same gate's "foreign domain" blocklist currently includes the `.in` web-address ending — so an India business on a `.in` email domain would be dropped as `foreign_cctld`. This is a **known rough edge**: today it fails *closed* (safe — it drops rather than risks a wrong send), and it needs reconciling (region-based check vs domain-based check) before India `.in` sending. Flag it; don't work around it.

Safety details at campaign-create time (so duplicates and clashes can't happen)

- **Collision scan:** before creating a campaign, the agent lists **every** existing campaign (all statuses, all pages) and **aborts** if the name matches a campaign it does not own. If it cannot read the full list, it **fails closed** (aborts) rather than guess.
- **Ledger-before-steps:** it records the new campaign id in its ledger *before* adding the email steps, so if a step call fails midway, the next run **resumes the half-built campaign** instead of creating a duplicate.
- **Re-scrub at first send:** a campaign staged days ago is **not** send-safe forever — lawful basis and the suppression list are **re-checked at the moment a human approves the first send**, and the one permitted change to a prior campaign is to **remove** (never add) someone who has since opted out.

Zoho fields that still have to be created

The current Zoho setup is missing fields the agent needs, so Vishal must create them before go-live: a working lead-status field (the built-in `Lead_Status` is empty, so the agent uses a new `LG_Lead_Status`), `LG_Disqualify_Recycle_Reason`, `Buyer_Role`, and an "accepted-by-sales" value — plus being aware that opt-out lives in different fields on Leads vs Contacts. In short: the lifecycle/role/reason machinery is designed but the fields **don't exist yet**.

Two different traffic-cops: "Conductor" vs "Router"

These are **not the same thing**:

- The **Conductor** is the (not-yet-live) orchestrator that would automatically hand seam events from the Growth Hacker to Lead Gen. Until it exists, **an operator polls the seam by hand**.
- The **Router** is a separate always-on service that moves prospects **between Woodpecker campaigns** based on clicks/replies. It is infrastructure outside this agent, and it only acts on the agent's own ledgered campaigns.

Why a "US" list can still show `region_unknown`

The geo-gate reads the `region`/`Country` **value on each row**, not the brief. If you sourced a US list but the rows themselves have no Country value filled in, the gate fails *closed* and marks them `region_unknown`. Fix: make sure **region is populated per row** on the clean Sheet — it is never inferred from "we meant US."

What to expect when you dry-run today

The mailbox id (`--mailbox`) and the approval token (`--approved`) the preflight asks for **do not exist yet** (no warmed mailbox is provisioned, and lawful basis is zero). So today you run the preflight **without** real values for those — and **HALT is the correct, expected result**. Seeing HALT means the wall is working, not that something is broken.

Where to run tests/preflight (Windows)

The toolkit lives at `iksula-marketplace/plugins/iksula-agents/tools/`. Open a terminal **in that folder** first so Python can find the `leadgen` package:

- `python -m unittest leadgen.tests.test_safety` (the 24 safety tests)
- `python -m leadgen.preflight --prospects leads.csv --template c0_body.txt --subject c0_subj.txt --source clean_sheet` (today: expect HALT). Note the preflight checks **one template at a time**, so validating a 3-touch C0→C1→C2 sequence means running it once per touch.

The scoring numbers are examples, not law

Any point values you saw (e.g. pricing-page +25, demo +40, fit-A floor 80) are **illustrative placeholders** from the scoring framework's "founder confirms" column. They are **not ratified** and must be signed off by DJ/Vishal before they drive anything.

Quick glosses for a few terms used above

- **SLA** — "service-level agreement": a promised time/standard (e.g. "respond to a hot lead within ~5 minutes").
- **MEDDIC / BANT / CHAMP** — named checklists salespeople use to confirm a deal is real (budget, decision-maker, need, timing, etc.).
- **GMV** — "gross merchandise value": the total sales value flowing through a business — a size signal.
- **MPP** — "Apple Mail Privacy Protection": Apple auto-opens emails, which fakes ~50% of "opens", which is why opens are ignored and clicks/replies are trusted.
- **CTOR vs CTR** — CTOR (click-to-open rate) = clicks among people who opened; CTR = clicks among everyone sent. CTOR is the more honest measure.
- **DKIM / DMARC** — email "ID checks" that prove a message really came from your domain; they protect deliverability (whether email lands in the inbox vs spam).
- **Single-threaded** — only one person at the target company is engaging; the agent caps such accounts below MQL until more of the buying group is involved.